RFID

Development Documentation

[V2.19]

Table of contents

1. Common methods and processes 6
1.1 List 6
1.2 Description 6
1.3 Process 6
2. RFID API 9
2.1 Initialization and connection 9
2.1.1 All-in-one machine and UART device 9
2.1.2 Bluetooth device initialization (for non-Bluetooth devices, please ignore this chapter) 10
2.1. 3 isConnected [Determine whether the RFID module is connected] 11
2.2 RFID tag inventory and recall 12
2.2.1 startInventoryWithTimeout [Start Inventory] 12
2.2.2 startInventory [Start Inventory] 14
2.2.3 registerCallback [Register the callback of inventory results] 15
2.2.4 stopInventory [Stop Inventory] 17
2.2.5 scanRfid [Query the tag once] 17
2.2.6 inventorySingle [Single tag query] 17
2.3 Inventory mode selection 18
2.3.1 setInventorySceneMode [Set inventory scene mode] 18
2.3.2 getInventorySceneMode[Get inventory scene mode] 19
2.4 Inventory storage bank settings 20
2.4.1 setQueryMemoryBank[Set the query storage bank mode] 20
2.4.2 getQueryMemoryBank[Get the mode of inventory bank] 23
2.5 Inventory Parameter Configuration 24
2.5.1 setOutputPower [Set output power] 24
2.5.2 getOutputPower [Query output power] 25
2.5.3 setProfile [Set BLF, Miller, Tari, PIE] 25
2.5.4 getProfile [Set BLF, Miller, Tari, PIE] 27
2.5.5 setInventoryWithSession 27
2.5.6 getInventoryWithSession 28
2.5.7 setInventoryWithTarget [Set the inventory Target] 29
2.5.8 getInventoryWithTarget [Get the target value of the inventory session] 30
2.5.9 setInventoryWithStartQvalue [Set the initial Q-value for inventory counting] 31
2.5.10 getInventoryWithStartQvalue [Get the initial Q-value when taking inventory] 32
2.5.11 setInventoryWithPassword【Set the access password】 32
2.5.12 getInventoryWithPasssword【Get the access password】 33
2.5.13 setInventoryRssiLimit [RSSI filtering] 33
2.5.14 isSupportInventoryRssiLimit [Whether to support limiting RSSI] 34

2.5.15 setRssiInDbm[Set the type of inventory return signal] 35
2.5.16 addMask [Add mask] 35
2.5.17 clearMask [Clear Mask] 38
2.6 Tag Operation 38
2.6.1 readTag [Read Tag] 39
2.6.2 writeTag [Write Tag] 40
2.6.3 writeTagEpc [Write tag EPC bank] 42
2.6.4 writeEpc [Randomly rewriting a tag's EPC] 43
2.6.5 lockTag [Lock Tag] 44
2.6.6 killTag [Kill Tag] 46
2.6.7 readDataByTid [Read data by tid] 47
2.6.8 writeTagByTid [Write data to each storage bank by tid] 48
2.6.9 lockByTID [Lock tag by TID] 50
2.6.10 killTagByTid [Kill tag by TID] 52
2.6.11 writeTagEpcByTid [Write EPC bank by TID] 53
2.6.12 maskReadTag [Read tag with mask] 53
2.6.13 maskWriteTag [Write tag with mask] 57
2.6.14 readTagExt [Read large capacity tags] 62
2.6.15 writeTagExt [Write large capacity tags] 64
2.6.16 eraseTag [Erase Tag] 65
2.6.17 findEpc [Search Tag] 67
2.6.18 lightUpLedTag [Light up the specified LED tag] 67
2.6.19 startInventoryLed [Query ELD tag] 68
2.6.20 stopInventoryLed [Stop querying ELD tags] 69
2. 7 Frequency band and other settings 70
2.7.1 setWorkRegion [Set regional frequency band] 70
2.7.2 getWorkRegion [Get working frequency] 70
2.7.3 setFrequencyRegion [Set frequency region] 71
2.7.4 getFrequencyRegion [Get frequency region] 72
2.7.5 getFirmwareVersion [Get firmware version] 73
2.7.6 setTagFocus [Set TagFocus command] 74
2.7.7 getTagFocus [Read TagFocus command] 74
2.7.8 getModuleFirmware [Get module firmware information] 75
2.7.9 GetReaderType [Get module type] 75
2.7.10 getReaderTemperature [Query internal temperature] 75
2.7.11 setCustomRegion [Set custom frequency band] 76
2.7.12 getCustomRegion [Get custom frequency band] 77
2.7.13 getEx10Version [Obtain E-Series chip information] 77
2.7.14 enableLog [enable or disable log] 78
2. 8 Buzzer setting 79
2.8.1 setBeepEnable [Set to enable the buzzer] 79
2.8.2 setBeepVolume [Set the buzzer volume] 79
2. 8 .3 startBeep [control the buzzer to sound once] 80

3 Firmware Upgrade 81
3.1 All-in-one device upgrade (including RFG91 UART) 81
3.1.1 updateReaderFirmwareByFile [RFID module upgrade - by reading files] 81
3.1.2 updateReaderFirmwareByByte [RFID module upgrade--through byte stream] 83
3.1.3 updateEx10ChipFirmwareByFile [RFID chip upgrade - by reading files] 85
3.1.4 updateEx10ChipFirmwareByByte [RFID chip upgrade - through byte stream] 86
3.2 Bluetooth device upgrade 88
3.2.1 updateBLEReaderFirmwareByFile [RFID module upgrade - by reading files] 88
3.2.2 updateBLEReaderFirmwareByByte [RFID module upgrade--by reading files] 90
3.2.3 updateBLEEx10ChipFirmwareByFile [RFID module upgrade - through byte stream] 91
3.2.4 updateBLEEx10ChipFirmwareByByte [RFID module upgrade--through byte stream] 93
4. Bluetooth and UART modular device interface description (non-integrated device) 96
4.1 Obtaining the results of scanning the barcode 96
4.2 Device status callback monitoring 97
4.2.1 setKeyEventListener [Register key event listener] 97
4.2.2 setBatteryGripListener [register device charging status, battery charge change monitoring] 98
4.3 setSwitchCallback [Monitor the switching between RFID modules] 100
4.3 startScanBarcode [Start scanning barcode] 101
4.4 getElectricQuantity [Get battery power] 102
4.5 getIsChange [Retrieve device charging status.] 102
4.6 getVersionSystem [Get device version] 103
4.7 getVersionBLE [Get Bluetooth version] 103
4.8 getDeviceSN [Get device SN] 104
4.9 setBeepRange [Set buzzer volume] 104
4.10 getBeepRange [Get the buzzer volume] 105
4.11 setSleepTime [Set sleep time] 105
4.12 getSleepTime [Get sleep time] 106
4. 1 3 setPowerOffTime [Set the timeout shutdown time] 106
4.14 getPowerOffTime [Get buzzer volume] 107
4.15 getBLEMac [Get the MAC address of the Bluetooth device] 107
4.16 setOfflineModeOpen [Set offline mode] 108
4.17 getOfflineModeOpen [Get the offline mode open status] 108
4.18 setOfflineTransferClearData [Set whether to automatically clear data

after offline mode transfer] 109
4.19 getOfflineTransferClearData [Get the status of automatic data clearing in offline mode] 109
4 . 2 0 setOfflineTransferDelay [Set the transmission time interval between data in offline mode] 110
4.2 1 getOfflineTransferClearData [Get the status of automatic data clearing in offline mode] 110
4.22 getOfflineQueryNum [Get the number of offline data] 111
4.23 getOfflineQueryMem [Get the memory percentage occupied by offline data] 111
4.24 offlineManaulClearData [Actively clear data saved in offline mode] 112
4.25 offlineStartTransferRFID [Actively clear RFID data saved in offline mode] 112
4.26 offlineStartTransferScan [Actively clear RFID data saved in offline mode] 113
4.27 modeResetFactory [Device firmware restores factory settings] 113
Appendix 1: Common error codes 115
Appendix 2: Frequency band description 117
Appendix 3: RSSI parameter comparison table 119
Appendix 4: Regional Comparison Table 121
Appendix 5: APP packaging obfuscation rules 123
Remark 124
Additional API Notes 124

1. Common methods and processes

1.1 List

- **Hardware requirements:** Make sure the equipment and RFID modules are provided by our company **.**
- **SDK content :** contains an aar package file, URFIDLibrary-xxxxxx.aar, which communicates with the hardware and provides a set of easy-to-develop interfaces for the application layer to call;
- **Demo:** UHFDemo is a demo example using ultra-high frequency, which can be used for reference .
- **Document :** " RFID Development Document v2.19.docx " is the SDK development guide and interface method instructions.

1.2 Description

RFID SDK for Android platform, development package URFIDLibrary-xxxxxx.aar, developers can directly call RFID module through API methods in SDK, which is efficient and simple. After successful initialization, the interface methods can be used normally .

1.3 Process

Integration steps:

1. **Add dependencies**

Import URFIDLibrary-xxxxxx.aar To your Android project :

```
implementation files ( 'libs/URFIDLibrary-xxxxxx.aar' )
```

2. **Initialize SDK**

Note: SDK methods generally block the main thread, so it is recommended to call the API in an asynchronous thread and then send a message to the main thread to update the UI. At the same time, the APIs need to be called in sequence, and the next instruction should be executed after the previous API is executed.

- When entering the APP for the first time , call the initialization method: init(Context context,InitListener listener) to enter the standby state (low power consumption) :
  RFIDSDKManager.getInstance().int init(Context context,InitListener listener);

- When exiting the application, call the method to disconnect the RFID module and enter a completely inoperative state. In this state, the RFID module does not consume power. To use it again next time, you need to call init(Context context, InitListener listener) again :

  RFIDSDKManager.getInstance(). release ();

4. Inventory **operation**

- Start inventory :
  RFIDSDKManager.getInstance().getRfidManager().
  startInventoryWithTimeout(int timeout)
  or
  RFIDSDKManager.getInstance().getRfidManager().startRead();
  (The inventory state consumes a lot of power. If the work is completed, please call the method to stop the inventory in time)
- Stop inventory :
  RFIDSDKManager.getInstance().getRfidManager().stopInventory();
- Inventory result data callback :
  RFIDSDKManager.getInstance().getRfidManager().registerCallback(new IRfidCallback() {
  @Override
  public void onInventoryTag(String EPC , String Data, String rssi) {}
  @Override
  public void onInventoryTagEnd() {}});
- Callback method and parameter description :

1> onInventoryTag [result callback method]

definition	void onInventoryTag (String EPC , String Data , int RSSI);		
illustrate	This method returns inventoried tag information via callback after inventory is initiated.		
parameter	name	type	Remark
	EPC	String	EPC of inventoried tags (excluding CRC and PC)

	Data	String	In hybrid query mode, it returns data from specified memory regions (e.g., TID or USER bank). Hybrid query mode is configured using setQueryMemoryBank(QueryMemBank mode)."
	RSSI	int	The signal strength of the tag . The value range is greater than 0. If you want to convert it to dBm value, you can refer to [Appendix 3: RSSI Parameter Comparison Table] or use the tool class conversion: int dBm = RssiToDBmUtils.dBmConver(int RSSI);

2> onInventoryTagEnd [Inventory end callback]

definition	void onInventoryTagEnd ();
illustrate	This method is called when the inventory is finished.

2. RFID API

2.1 Initialization and connection

2.1.1 All-in-one machine and UART device

2.1.1.1 init [RFID Initialization]

definition	void init (Context context , InitListener listener);		
illustrate	RFID Initialization		
parameter	name	type	Remark
	context	Context	Application Context

	listener	InitListener	Listen for callbacks of initialization success or failure
			`new InitListener() {`
			`@Override`
			`public void onStatus( boolean status) {`
			`}`
			`}`
			`status :`
			`t rue: Initialization successful`
			`false : Initialization failed`

return	none.		
Reference code	none.		

2.1.1.2 release [disconnect module]

definition	void release ();
illustrate	Disconnect the RFID module to enter a completely inactive state; this function must be called upon application exit.

parameter	name	type	Remark
return	none.		
Reference code	none.		

2.1.2 Bluetooth device initialization (for non-Bluetooth devices, please ignore this chapter)

2.1.2.1 initBTToMac [Bluetooth device initialization]

definition	void initBTtoMac (Context context , String mac , InitListener listener);		
illustrate	Connect to Bluetooth devices with RFID function .		
parameter	name	type	Remark
	context	Context	Context
	Mac	String	The MAC address of the Bluetooth device to be connected, for example: 41:BB:00:F8:DB:51

	listener	InitListener	Listen for callbacks of initialization success or failure
			new InitListener() {
			@Override
			public void onStatus(boolean status) {
			}
			}
			status :
			true: Initialization (connection) successful
			false : Initialization (connection) failed

return	none		

Reference code	none.

2.1.2.2 release [Release resources]

definition	void release ();		
illustrate	Release resources and disconnect the Bluetooth device.		
parameter	name	type	Remark
return	none		
Reference code	none		

2.1. 3 isConnected [Determine whether the RFID module is connected]

definition	boolean isConnected ();		
illustrate	Determine whether the RFID module is connected .		
parameter	name	type	Remark
return (boolean)	true : connected ; false : Not connected .		

Reference code	none

2.2 RFID tag inventory and recall

After the device is initialized, call the following RfidManager to control subsequent APIs :

```
RFIDSDKManager.getInstance().getRfidManager()
```

2.2.1 startInventoryWithTimeout [Start Inventory]

definition	int startInventoryWithTimeout (int timeout);		
illustrate	Start the inventory, the timeout parameter here is in seconds . When the inventory execution reaches the timeout time, the inventory will be automatically stopped, and a callback notification will be sent in the onInventoryTagEnd() callback method. If the timeout time has not been reached and you want to stop it, you can also call 2.2.4 stopInventory() to stop it manually. The result is returned in the IRfidCallback() callback.		
parameter	name	type	Remark
	timeout	int	Timeout period , in seconds, must be greater than or equal to 0 and cannot exceed 36000 seconds: Setting 0: Inventory process will run indefinitely and must be explicitly stopped by calling stopInventory(). Setting 0 < timeout <= 36000: The inventory process will automatically cease upon reaching

<table>
<tr><td></td><td></td><td></td><td>the specified timeout duration. It may also be manually terminated prematurely by invoking stopInventory() during its execution.</td></tr>
<tr><td>return
(int)</td><td colspan="3">Success: RfidErrorConstants.SUCCESS;
Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .</td></tr>
<tr><td>Reference code</td><td colspan="3">RFIDSDKManager.getInstance().getRfidManager().startInventoryWithTimeout(300);

If you want to monitor the buttons of the RFID controller to trigger the inventory method, you can use the following method:
1. All-in-one device, listen to Android standard key events:

@Override
public boolean dispatchKeyEvent(KeyEvent event) {
if ((event.getKeyCode() == 523 || event.getKeyCode() == 515)
&& event.getRepeatCount() == 0) {
//TODO The key value of the PTT button on the left side of DT610 is 515
if (event.getAction() == KeyEvent.ACTION_DOWN) {
//start Inventory
//RFIDSDKManager.getInstance().getRfidManager().startInventoryWithTimeout(300);
return true;
} else if (event.getAction() == KeyEvent.ACTION_UP) {
//stop Inventory
//RFIDSDKManager.getInstance().getRfidManager().stopInventory();
return true;
}
}
return super.dispatchKeyEvent(event);
}

2. Bluetooth device, callback of monitoring method
GripDeviceManager.getInstance().setKeyEventListener(new KeyEventListener() {
@Override
public void event(int keyCode, boolean isDown) {
//keycode == BTKeyEvent.BT_SCAN The handle button was pressed
//isDown true: down false: up
if (keyCode == BTKeyEvent.BT_SCAN) {
if (isDown) {
//start Inventory</td></tr>
</table>

	//RFIDSDKManager.getInstance().getRfidManager().startInventoryWithTimeout(300);
	} else {
	//stop Inventory
	//RFIDSDKManager.getInstance().getRfidManager().stopInventory();
	}
	}
	}
	});

2.2.2 startInventory [Start Inventory]

definition	int startInventory ();		
illustrate	Start inventory. Once started, it must be stopped exclusively by calling stopInventory(). Tags identified during inventory are returned via the IRfidCallback() callback interface.		
parameter	name	type	Remark
return (int)	Success: RfidErrorConstants.SUCCESS; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		

Reference code	RFIDSDKManager.getInstance().getRfidManager().startInventory();
	
	If you want to monitor the buttons of the RFID controller to trigger the inventory method, you can use the following method:
	1. All-in-one device, listen to Android standard key events:
	
	@Override
	public boolean dispatchKeyEvent(KeyEvent event) {
	if (event.getKeyCode() == 523 && event.getRepeatCount() == 0) {
	//TODO The Grip key value of the key is 523
	if (event.getAction() == KeyEvent.ACTION_DOWN) {
	//start Inventory
	//RFIDSDKManager.getInstance().getRfidManager().startInventory();
	return true;
	} else if (event.getAction() == KeyEvent.ACTION_UP) {		

	//stop Inventory
	//RFIDSDKManager.getInstance().getRfidManager().stopInventory();
	return true;
	}
	}
	return super.dispatchKeyEvent(event);
	}
	
	2. Bluetooth device, callback of monitoring method
	GripDeviceManager.getInstance().setKeyEventListener(new KeyEventListener() {
	@Override
	public void event(int keyCode, boolean isDown) {
	//keycode == BTKeyEvent.BT_SCAN The handle button was pressed
	//isDown true: down false: up
	if (keyCode == BTKeyEvent.BT_SCAN) {
	if (isDown) {
	//start Inventory
	//RFIDSDKManager.getInstance().getRfidManager().startInventory();
	} else {
	//stop Inventory
	//RFIDSDKManager.getInstance().getRfidManager().stopInventory();
	}
	}
	}
	});

2.2.3 registerCallback [Register the callback of inventory results]

definition	void registerCallback (IRfidCallback cb);
illustrate	Register callback (tag data will be called back in the registered callback).

parameter	name	type	Remark
	cb	IRfidCallback	Callbacks
return (void)	none		
Reference code	none		

2.2.3.1 onInventoryTag ()

definition	void onInventoryTag (String EPC , String Data , int RSSI);		
illustrate	After starting the inventory, this method callbacks the inventoried tag information.		
parameter	name	type	Remark
	EPC	String	Inventoried tag's EPC (excluding CRC and PC)
	Data	String	Data returned from corresponding memory banks in Hybrid Query Mode, e.g., TID bank data, USER bank data, or Reserved bank data. Hybrid Query can be configured via the setQueryMemoryBank(QueryMemBank mode) method.
	RSSI	int	Tag's RSSI (Received Signal Strength Indication) . The value range is a value greater than 0. If you want to convert it to dBm value, please refer to [Appendix 3: RSSI Parameter Comparison Table]

2.2.3.2 onInventoryTagEnd

definition	void onInventoryTagEnd ();
illustrate	This method is called when the inventory is finished.

2.2.4 stopInventory [Stop Inventory]

definition	int stopInventory (); 		
illustrate	Stop inventory.		
parameter	name	type	Remark
return (int)	Success:RfidErrorConstants.SUCCESS; Failure: (For detailed error code information, reference: <Appendix 1: Common error codes>) .		
Reference code	none		

2.2.5 scanRfid [Query the tag once]

definition	void scanRfid (); 		
illustrate	Executes a single inventory cycle that automatically terminates after completion. This may detect zero or more tags, with results returned through the IRfidCallback() interface.		
parameter	name	type	Remark
return (void)			
Reference code	none		

2.2.6 inventorySingle [Single tag query]

definitio	int inventorySingle ();

n			
illustrate	Each invocation retrieves at most one tag. If no tags are present in the interrogation zone, the operation may yield no result. Results are returned through the IRfidCallback() interface.		
parameter	name	type	Remark
return	RfidErrorConstants.SUCCESS; Others : The call failed (for error code information, reference: <Appendix 1: Common Error Codes>) . The reading result is returned in the onInventoryTag() callback.		
Reference code	none.		

2.3 Inventory mode selection

2.3.1 setInventorySceneMode [Set inventory scene mode]

definition	int setInventorySceneMode (InventorySceneMode mode);
illustrate	Pre-configured operational profiles for specific application scenarios are provided. Select the appropriate mode for your use case. For custom configurations, utilize the parameter settings outlined in **Section 2.5 Inventory Parameter Configuration ** of this document.

	Note: - Parameters *Session*, *Qvalue*, *Target*, and *Profile* are **locked** in non-custom modes (`InventorySceneMode.CUSTOM_MODE`). - Output power reverts to the mode's default value after mode selection. Reapply custom power settings using		
parameter	name	type	Remark
	mode	InventorySceneMode	For **Inventory Scene Modes**, refer to the **Inventory Scene Mode Descriptions** section below.
return (int)	Success: RfidErrorConstants.SUCCESS; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	RFIDSDKManager.getInstance().getRfidManager().set InventorySceneMode (InventorySceneMode.BATTERY_LIFE);		

Inventory scene mode description:

Inventory of scene modes (InventorySceneMode)	Applicable scenarios	Core Features
BATTERY_LIFE	Low power consumption scenario	Extend device battery life
FASTEST_READ	Quickly read the entire Tag	High-speed inventory, suitable for E series chips
REPEAT_READ	Ranging and multi-tag statistics	Performs rapid repeated reads and accumulates tag counts
BALANCED_PERFORMANCE	Balanced mode	Balancing performance and battery life
CUSTOM_MODE	Custom parameter settings	Parameters can be configured by yourself

2.3.2 getInventorySceneMode[Get inventory scene mode]

definition	Inventory Scene Mode getInventorySceneMode () ;

illustrate	Retrieves the previously configured inventory scene mode.		
parameter	name	type	Remark
return (InventorySceneMode)	InventorySceneMode : InventorySceneMode.BATTERY_LIFE InventorySceneMode.FASTEST_READ InventorySceneMode.REPEAT_READ InventorySceneMode.BALANCED_PERFORMANCE InventorySceneMode.CUSTOM_MODE		
Reference code	InventorySceneMode mode = RFIDSDKManager.getInstance().getRfidManager(). g et InventorySceneMode ();		

2.4 Inventory storage bank settings

2.4.1 setQueryMemoryBank[Set the query storage bank mode]

definition	int setQueryMemoryBank(QueryMemBank mode , int startAdd, int readLen);		
illustrate	Set the storage bank for querying labels during inventory checks.		
parameter	name	type	Remark
	mode	QueryMemBank	See the description of QueryMemBank below .
	startAdd	int	The starting address of the memory bank to inventory, specified in word addresses (1 word = 16 bits):
	readLen	int	The **word count** (1 word = 16 bits) to read during inventory:

return (int)	Success:RfidErrorConstants.SUCCESS; Failure: (For detailed error code information, reference: <Appendix 1: Common error codes>) .		
Reference code	RFIDSDKManager.getInstance().getRfidManager().setQueryMemoryBank (QueryMemBank.EPC,0,0); RFIDSDKManager.getInstance().getRfidManager().setQueryMemoryBank (QueryMemBank.EPC_USER,startAddress,readLen);		

QueryMemBank Description:

QueryMemBank	Read Bank	illustrate
EPC	EPC Query	set both `startAddress` and `readLen` to `0` to read the **default EPC length** (entire EPC bank excluding CRC/PC bits).
EPC_TID	EPC+TID Bank Query	Performs **simultaneous EPC and TID bank queries**: - EPC bank: Read at **default length** (full EPC payload, **excluding CRC and PC bits**) - TID bank: Data returned via `onInventoryTag(String EPC, String Data, int RSSI)` callback: - `Data` field contains TID values - Reads from `startAddress` (word address) - Length = `readLen` words (1 word = 16 bits)
EPC_USE	EPC+USER Bank Query	Performs **simultaneous EPC and USER bank queries**: - EPC bank: Read at **default length** (full EPC payload, **excluding CRC and PC bits**) - USER bank: Data returned via `onInventoryTag(String EPC, String Data, int

		RSSI)` callback: - `Data` field contains USER memory values - Reads from `startAddress` (word address) - Length = `readLen` words (1 word = 16 bits)
EPC_RESERVED	EPC+RESERVED（Password） Bank Query	Performs **simultaneous EPC and RESERVED bank queries**: - EPC bank: Read at **default length** (full EPC payload, **excluding CRC and PC bits**) - RESERVED bank: Data returned via `onInventoryTag(String EPC, String Data, int RSSI)` callback: - `Data` field contains RESERVED (Password) values - Reads from `startAddress` (word address) - Length = `readLen` words (1 word = 16 bits)
FASTTID	FASTTID Query	**Impinj FastTID Mode**: Enables rapid reading of EPC and TID data for tags supporting FastTID technology during inventory operations. - TID data is returned via `onInventoryTag(String EPC, String Data, int RSSI)` callback: - `Data` field contains full TID content - Always reads from **address 0** - Fixed length: **6 words (96 bits)**
EPC_EPC	EPC+EPC Bank Query	Performs **dual EPC bank queries**: - **First EPC read**: Default length (full EPC payload, excluding CRC/PC bits) - **Second EPC read**: Custom read returned

		via `onInventoryTag(String EPC, String Data, int RSSI)` callback: - `Data` field contains second EPC read values - Reads from `startAddress` (word address) - Length = `readLen` words (1 word = 16 bits)

2.4.2 getQueryMemoryBank[Get the mode of inventory bank]

definition	QueryMemBank getQueryMemoryBank( ) ;		
illustrate	The pattern for obtaining the inventory data bank。		
parameter	name	type	Remark

return (int)	value returned :
	QueryMemBank queryMemBank :
	
	queryMemBank.getStartAddress(); //Read Start Address （1 Word = 16 bits）
	queryMemBank.getReadLength(); //Read Word Length (1 Word = 16 bits)		

Reference code	QueryMemBank queryMemBank = RFIDSDKManager.getInstance().getRfidManager().getQueryMemoryBank();
	int startAdd = queryMemBank.getStartAddress();
	int readLen = queryMemBank.getReadLength();		

2.5 Inventory Parameter Configuration

2.5.1 setOutputPower [Set output power]

definition	int setOutputPower (int outputPower);		
illustrate	Setting Power		
parameter	name	type	Remark
	outputPower	int	Output power of the equipment The maximum power value supported by the module can be obtained through the following method: int maxPower = RFIDSDKManager.getInstance().getRfidManager().getSupportMaxOutputPower(); maxPower represents the maximum supported power value.
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.5.2 getOutputPower [Query output power]

definition	int getOutputPower (); 		
illustrate	Query the output power.		
parameter	name	type	Remark
return (int)	Returns the output power (failed to get if less than 0)		
Reference code	none		

2.5.3 setProfile [Set BLF, Miller, Tari, PIE]

definition	int setProfile (RfidProfile profile);
illustrate	Configure Link Parameters: BLF (Backscatter Link Frequency), Miller, Tari, PIE。 This method uses a predefined combination of BLF, Miller, Tari, and PIE parameters. Note: If the device is in non-custom mode (InventorySceneMode.CUSTOM_MODE), this parameter cannot be configured. To modify it, first set the inventory mode to Custom Mode (InventorySceneMode.CUSTOM_MODE) via 2.3.1

	setInventorySceneMode() .		
parameter	name	type	Remark
	param	int	The chip type can be obtained through 2.7.13 getEx10Version [Get Ex10 chip information] . For compatible RFID Profile combinations across different chips, refer to the table below.
Returns (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	RFIDSDKManager.getInstance().getRfidManager().setProfile(RfidProfile.PROFILE_5);		

Supported RfidProfiles for Different Chips:

RfidProfile	Chip support	Parameter combination
PROFILE_3	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 20.0μs, BLF: 320kHz, Miller=2
PROFILE_5	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 20.0μs, BLF: 320kHz, Miller=4
PROFILE_7	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 20.0μs, BLF: 250kHz, Miller=4
PROFILE_12	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 15.0μs, BLF: 320kHz, Miller=2
PROFILE_13	E710, E510, E310	PR_ASK, PIE: 2.0, Tari: 20.0μs, BLF: 160kHz, Miller=8
PROFILE_1	E710, E510	PR_ASK, PIE: 2.0, Tari: 7.5μs, BLF: 640kHz, Miller=2
PROFILE_15	E710, E510	PR_ASK, PIE: 2.0, Tari: 7.5μs, BLF: 640kHz, Miller=4
PROFILE_51	E710, E510	DSB, PIE: 1.5, Tari: 6.25μs, BLF: 640, Miller=2
PROFILE_53	E710, E510	PR_ASK, PIE: 1.5, Tari: 7.5, BLF:640kHz, Miller=4
PROFILE_11	E710	PR_ASK, PIE: 2.0, Tari: 7.5μs, BLF: 640kHz, FM0
PROFILE_50	E710	DSB, PIE: 1.5, Tari: 6.25μs, BLF: 640kHz, FM0
PROFILE_52	E710	PR_ASK, PIE: 2.0, Tari: 15.0μs,

		BLF: 426kHz, FM0

2.5.4 getProfile [Set BLF, Miller, Tari, PIE]

definition	RfidProfile getProfile () ;		
illustrate	Get Current Link Parameter Set: BLF (Backscatter Link Frequency), Miller, Tari, PIE		
parameter	name	type	Remark
Returns (RfidProfile)			
Reference code	RfidProfile rfidProfile = RFIDSDKManager.getInstance().getRfidManager().getProfile();		

2.5.5 setInventoryWithSession

definition	int setInventoryWithSession (int session);		
describe	Set up the inventory session Note: If the device is in non-custom mode (InventorySceneMode.CUSTOM_MODE), this parameter cannot be configured. To modify it, first set the inventory mode to Custom Mode (InventorySceneMode.CUSTOM_MODE) via 2.3.1 setInventorySceneMode() .		
parameter	name	type	Remark
	session	Session	Session defaults to SESSION.S1 ;

			- SESSION.S0 : 0 - SESSION.S1 : 1 - SESSION.S2 : 2 - SESSION.S3 : 3 SESSION.S0: Designed for querying individual or small tag populations, achieving higher per-tag read rates within identical time intervals. SESSION.S1: Optimized for dynamic environments with large tag populations, prioritizing discovery of new tags. SESSION.S2/SESSION.S3: Tailored for static tag inventories leveraging persistent session states.
Return value (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	RFIDSDKManager.getInstance().getRfidManager().setInventoryWithSession(SESSION.S1);		

2.5.6 getInventoryWithSession

definition	int getInventoryWithSession ();		
describe	Get the inventory session .		
parameter	name	type	Remark
Return value	success: - SESSION.S0 : 0		

(int)	- SESSION.S1 : 1 - SESSION.S2 : 2 - SESSION.S3 : 3 Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .
Reference code	int session = RFIDSDKManager.getInstance().getRfidManager().getInventoryWithSession();

2.5.7 setInventoryWithTarget [Set the inventory Target]

definition	int setInventoryWithTarget (int target);		
describe	Set the Target for the Inventoried Flag state to filter responding tags. Default: Target.TARGET_AB; Note: If the device is in non-custom mode (InventorySceneMode.CUSTOM_MODE), this parameter cannot be configured. To modify it, first set the inventory mode to Custom Mode (InventorySceneMode.CUSTOM_MODE) via 2.3.1 setInventorySceneMode() .		
parameter	name	type	Remark
	target	Target	- `Target.TARGET_A` - 0: Only tags in **State A** respond to inventory (`Inventoried Flag = A`) - `Target.TARGET_B` - 1: Only tags in **State B** respond to inventory (`Inventoried Flag = B`)

			- `Target.TARGET_AB` - 2: **Dynamically toggles states**: • First query: Tags in **State A** respond • Subsequent queries: Automatically switches to **State B** tags
Return value (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code			

2.5.8 getInventoryWithTarget [Get the target value of the inventory session]

definition	int getInventoryWithTarget ();		
describe	Get the inventory Target value.		
parameter	name	type	Remark
Return value (int)	success: 0 - TARGET_A; 1 - TARGET_B; 2 - TARGET_AB . Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code			

2.5.9 setInventoryWithStartQvalue [Set the initial Q-value for inventory counting]

<table>
<tr><td>definition</td><td colspan="3">int setInventoryWithStartQvalue (int QvalueStart);</td></tr>
<tr><td>describe</td><td colspan="3">Set the initial Q-value when taking inventory .

Note:
If the device is in non-custom mode (InventorySceneMode.CUSTOM_MODE), this parameter cannot be configured.

To modify it, first set the inventory mode to Custom Mode (InventorySceneMode.CUSTOM_MODE) via 2.3.1 setInventorySceneMode() .</td></tr>
<tr><td rowspan="2">parameter</td><td>name</td><td>type</td><td>Remark</td></tr>
<tr><td>QvalueStart</td><td>Int</td><td>Initial Q-value determines the number of time slots in the first inventory round. It dynamically adapts based on detected tag responses during operation. Valid range: 0-15 (default: 6).

Lower Q-values are recommended for sparse tag populations, while higher values suit dense tag clusters.

Note: Q operates as an adaptive algorithm - it automatically increases/decreases during inventory according to real-time tag density.</td></tr>
<tr><td>Return value (int)</td><td colspan="3">Success: RfidErrorConstants.SUCCESS ;
Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .</td></tr>
</table>

Reference code	

2.5.10 getInventoryWithStartQvalue [Get the initial Q-value when taking inventory]

definition	int getInventoryWithStartQvalue ();		
describe	Get Initial Q-value for Inventory		
parameter	name	type	Remark
Return value (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code			

2.5.11 setInventoryWithPassword【Set the access password】

definition	int setInventoryWithPassword(String password);		
describe	Set Inventory Access Password. Default: "00000000"		
parameter	name	type	Remark
	password	String	The access password of the tag is 2 word and in hexadecimal string format.
Return value (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference			

code	

2.5.12 getInventoryWithPasssword【Get the access password】

definition	String getInventoryWithPassword();		
describe	Get Inventory Access Password.		
parameter	name	type	Remark
Return value (String)	Return the access password during the inventory.		
Reference code			

2.5.13 setInventoryRssiLimit [RSSI filtering]

definition	int setInventoryRssiLimit (int rssi);
describe	Filter tags with RSSI values below the set threshold. In such scenarios, simultaneously reducing transmit power (typically below 20 dBm) can achieve this effect. Before implementation, verify device support using: 2.5.12 isSupportInventoryRssiLimit() Applies only during inventory operations. To disable the RSSI filter, set the threshold to 0. Note: If the device is in non-custom mode (InventorySceneMode.CUSTOM_MODE), this parameter cannot be configured.

	To modify it, first set the inventory mode to Custom Mode (InventorySceneMode.CUSTOM_MODE) via 2.3.1 setInventorySceneMode() .		
parameter	name	type	Remark
	rssi	int	Filter tags with RSSI values less than
Return value (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code			

2.5.14 isSupportInventoryRssiLimit [Whether to support limiting RSSI]

definition	int isSupport InventoryRssiLimit ();		
describe	Gets whether to support limiting RSSI during inventory counting.		
parameter	name	type	Remark
Return value (int)	Supported : true ; Not supported : false .		
Reference code			

2.5.15 setRssiInDbm[Set the type of inventory return signal]

definition	int setRssiInDbm(boolean isDBm);		
describe	The default is false. In the inventory result callback method onInventoryTag(String EPC, String Data, int rssi);, the rssiparameter defaults to a positive signal value greater than 0. To configure the system to output RSSI in dBm (decibel-milliwatts), the absolute unit for measuring RF signal strength, by default, this method can be set to true.		
parameter	name	type	Remark
	isDBm	boolean	true : Sets the return signal value in dBm (decibel-milliwatts), the absolute unit for measuring RF signal strength. false : Sets the return value to a positive numerical signal value greater than 0 (arbitrary scale).
Return value (int)			
Reference code			

2.5.16 addMask [Add mask]

definition	void addMask (int mem , int startAddress , int len , String data);
illustrat	Add filtering parameters for inventory operations. After

<table>
<tr><td>e</td><td colspan="3">configuring the mask parameter, the inventory process will only identify tags programmed with the corresponding mask, filtering out non-matching tags.
Multiple distinct mask data entries can be added successively.</td></tr>
<tr><td rowspan="3">parameter</td><td>name</td><td>type</td><td>Remark</td></tr>
<tr><td>mem</td><td>int</td><td>Filter bank:
TAG storage bank :
MemoryBank.RESERVED (RESERVED)
MemoryBank.EPC (EPC)
MemoryBank.TID (TID)
MemoryBank.USER (USER)</td></tr>
<tr><td>startAddress</td><td>int</td><td>Start Address Calculation: Note the parameter differs from other methods. Data indexing is based on character positions in the hexadecimal string:
EPC Memory Bank Example
EPC: E280FE9889895674
To mask "FE98" → Start Address = 12
(Reason: EPC includes CRC+PC prefix)

TID/USER Memory Bank Example
TID/USER: 00001234567898762345
To mask "56789876" → Start Address = 8
(No prefix, direct indexing from position 0)</td></tr>
</table>

	len	int	Mask Length Parameter: Note this differs from other methods. The value specifies the character length of the target hexadecimal string to filter, and must be an even number: EPC Memory Bank Examples EPC: E280FE9889895674 To mask "FE98" → Length = 4 *(4 hex characters = 2 bytes)* EPC: 00001234567898762345 To mask "567898762345" → Length = 12 *(12 hex characters = 6 bytes)*
	data	String	Filter data (even length)
return Void			
Reference code	Example: Tag Content: Epc:123456789 Tid:234567891 If you want to filter the TID bank starting from the second letter, you need to filter the data using 4567. You should set: memMask:MemoryBank.TID,startMask:2,lenMask：4， datasMask:4567.		

	The code:mRfidManager.addMask(MemoryBank.TID , 2,4 ,” 4567”)

2.5.17 clearMask [Clear Mask]

definitio n	void clearMask ();		
illustrat e	Clear Mask (Filter) Configuration.		
parameter	name	type	Remark
Void			
Reference code	none		

2.6 Tag Operation

In the Gen2 protocol:

1 word = 16 bits (2 bytes or 4 hexadecimal characters)

Example:

type	Legal input examples	illustrate

type	Legal input examples	illustrate
Example 1	`"E280"`	1 word length = 4 hexadecimal characters
Example 2	`"E2808080"`	2 words = 8 hexadecimal characters
Example 3	`"E2808080A001"`	3 words = 12 hexadecimal characters

2.6.1 readTag [Read Tag]

definition	TagResult readTag (String EPC , int memBank , int startAdd , int wordCnt , String password);		
illustrate	Read tag information.		
parameter	name	type	Remark
	EPC	Stirng	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of the hexadecimal string must be a multiple of 4).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	memBank	int	TAG Memory Banks: MemoryBank.RESERVED (Password Bank: First 2 Words = Kill Password, Last 2 Words = Access Password) MemoryBank.EPC (Electronic Product Code) MemoryBank.TID (Tag Identifier) MemoryBank.USER (User Memory)
	startAdd	int	Start Address of the target memory region

			for read/write operations, specified in Word Address units.
	wordCnt	int	Word Count to Read (When reading large content, iterative reads are recommended with a maximum of 32 Words per operation).
return (TagResult)	TagResult class field information: **code** : Status code: Success : RfidErrorConstants.SUCCESS ; Failure: Other (For error code information, reference: <Appendix 1: Common Error Codes>) . **message** : Error message. **data** : If code == RfidErrorConstants.SUCCESS, this field returns the read tag value.		
Reference code	none		

2.6.2 writeTag [Write Tag]

definition	int writeTag (String EPC , String password , int memBank , int startAdd , String writeData);		
illustrate	Write the tag.		
parameter	name	type	Remark
	EPC	String	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of the hexadecimal

			string must be a multiple of 4).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	memBank	int	TAG Memory Banks: MemoryBank.RESERVED (Password Bank: First 2 Words = Kill Password, Last 2 Words = Access Password) MemoryBank.EPC (Electronic Product Code) MemoryBank.TID (Tag Identifier) MemoryBank.USER (User Memory)
	startAdd	int	Starting position of the memory area to be read/written, in units of Word Address. Note that when writing to the EPC bank: 1>.If the EPC length needs to be changed, writing must start from the PC word (starting address 1). The calculated value of the PC word must be placed as the first word of the write data. (PC: Protocol Control, 协议控制位) 2>.If the EPC length does not need to change after writing, writing

			may start from starting address 2 directly.
	writeData	String	Data to be written, in hexadecimal string format. The data length must be an integer multiple of 1 Word length (i.e., the hexadecimal string length must be a multiple of 4). If the write content is large, it is recommended to perform cyclic writing, with a maximum length of 32 words per write operation.
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.6.3 writeTagEpc [Write tag EPC bank]

definition	int writeTagEpc (String EPC , String password , String writeData);
illustrate	Modify the EPC value of the tag (the PC value will be changed by default) , rewrite the original EPC value to the written EPC value, that is, change the tag EPC to the newly written one .

parameter	name	type	Remark
	EPC	String	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of the hexadecimal string must be a multiple of 4).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	writeData	String	Data to be written, in hexadecimal string format. The data length must be an integer multiple of 1 Word length (i.e., the hexadecimal string length must be a multiple of 4).
return (int)	Success: RfidErrorConstants.SUCCESS; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.6.4 writeEpc [Randomly rewriting a tag's EPC]

Definition 1	int writeEpc (String EPC , String password);
illustrate	Randomly rewriting a tag's EPC will simultaneously alter its PC value.

parameter	name	type	Remark
	EPC	String	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of the hexadecimal string must be a multiple of 4).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.6.5 lockTag [Lock Tag]

definition	int lockTag (String EPC , String password , i nt lockBank , int lockType);		
illustrate	Lock tag		
parameter	name	type	Remark
	EPC	String	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of

			the hexadecimal string must be a multiple of 4).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	lockBank	int	LockMemBank.KILL_PW - Controls the Kill password read/write protection settings. LockMemBank.ACCESS_PW - Controls the Access password read/write protection settings. LockMemBank.EPC - Controls the EPC memory read/write protection settings. LockMemBank.TID - Controls the TID memory read/write protection settings. LockMemBank.USER - Controls the User memory read/write protection settings.
	lockType	int	Configurable values: LockType.SET_READABLE_OR_WRITABLE LockType.SET_ALWAYS_READABLE_OR_WRITABLE LockType.SET_WITH_PASSWORD_READABLE_OR_WRITABLE LockType.SET_NEVER_READABLE_OR_WRITABLE parameter is different when the lock bank is different. For details, see the following table : **Relationship between**

			lockBank and lockType
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

The relationship between lock Bank and lock Type is as follows :

	LockMemBank.KILL_PW LockMemBank.ACCESS_PW	LockMemBank.EPC LockMemBank.TID LockMemBank.USER
LockType.SET_READABLE_OR_WRITABLE	Set to read and write	Set to writable
LockType.SET_ALWAYS_READABLE_OR_WRITABLE	Set to always read and write	Set to always writable
LockType.SET_WITH_PASSWORD_READABLE_OR_WRITABLE	Set to read and write with password	Set to writable with password
LockType.SET_NEVER_READABLE_OR_WRITABLE	Set to never be readable or writable	Set to never be writable.

2.6.6 killTag [Kill Tag]

definitio	int killTag (String EPC , String password);

n			
illustrate	Destroy tag (The kill password must not be the default password 00000000. If the kill password is set to the default 00000000, it must be modified to a non-default password before initiating destruction).		
parameter	name	type	Remark
	EPC	String	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of the hexadecimal string must be a multiple of 4).
	password	String	Kill Password, 2 words in length, hexadecimal string format.
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.6.7 readDataByTid [Read data by tid]

definition	String readDataByTid (String TID , int memBank , int startAdd , int wordCnt , String password);		
illustrate	Read data through the tag tid		
parameter	name	type	Remark
	TID	String	Tag's TID, in hexadecimal string

			format.
	memBank	int	TAG Memory Banks: MemoryBank.RESERVED (Password Bank: First 2 Words = Kill Password, Last 2 Words = Access Password) MemoryBank.EPC (Electronic Product Code) MemoryBank.TID (Tag Identifier) MemoryBank.USER (User Memory)
	startAdd	int	Start Address of the target memory region for read/write operations, specified in Word Address units.
	wordCnt	int	Word Count to Read (When reading large content, iterative reads are recommended with a maximum of 32 Words per operation).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
return String	Returns the read content if successful, otherwise returns null.		
Reference code	none		

2.6.8 writeTagByTid [Write data to each storage bank by tid]

definitio	int writeTagByTid (String TID , int memBank , int startAdd , String password ,

n	String writeData);		
illustrate	Write data to each storage bank.		
parameter	name	type	Remark
	TID	String	Tag's TID, in hexadecimal string format.
	memBank	int	TAG Memory Banks: MemoryBank.RESERVED (Password Bank: First 2 Words = Kill Password, Last 2 Words = Access Password) MemoryBank.EPC (Electronic Product Code) MemoryBank.TID (Tag Identifier) MemoryBank.USER (User Memory)
	startAdd	int	Start Address of the target memory region for read/write operations, specified in Word Address units.
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	writeData	String	Data to be written, in hexadecimal string format. The data length must be an integer multiple of 1 Word length (i.e., the hexadecimal string length must be a multiple of 4). If the write content is large, it is recommended to perform cyclic writing, with a maximum length of 32 words per write operation.
return	Success: RfidErrorConstants.SUCCESS ;		

(int)	Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .
Reference code	none

2.6.9 lockByTID [Lock tag by TID]

definition	int lockByTID (String TID , int lockBank , int lockType , String password);		
illustrate	Lock tag identified by TID.		
parameter	name	type	Remark
	TID	String	Tag's TID, in hexadecimal string format.
	lockBank	int	LockMemBank.KILL_PW - Controls the Kill password read/write protection settings. LockMemBank.ACCESS_PW - Controls the Access password read/write protection settings. LockMemBank.EPC - Controls the EPC memory read /write protection settings. LockMemBank.TID - Controls the TID memory read/write protection settings. LockMemBank.USER - Controls the User

			memory read/write protection settings.
	lockType	int	Configurable values: LockType.SET_READABLE_OR_WRITABLE LockType.SET_ALWAYS_READABLE_OR_WRITABLE LockType.SET_WITH_PASSWORD_READABLE_OR_WRITABLE LockType.SET_NEVER_READABLE_OR_WRITABLE parameter is different when the lock bank is different. For details, see the following table : **Relationship between lockBank and lockType**
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

The relationship between lockBank and lockType is as follows :

	LockMemBank.KILL_PW LockMemBank.ACCESS_PW	LockMemBank.EPC LockMemBank.TID LockMemBank.USER
LockType.SET_READABLE_OR_WRITABLE	Set to read and write	Set to writable
LockType.SET_ALWAYS_READABLE_OR_WRITABLE	Set to always read and write	Set to always writable

LockType.SET_WITH_PASSWORD_READABLE_OR_WRITABLE	Set to read and write with password	Set to writable with password
LockType.SET_NEVER_READABLE_OR_WRITABLE	Set to never be readable or writable	Set to never be writable.

2.6.10 killTagByTid [Kill tag by TID]

definition	int killTagByTid (String TID , String password);		
illustrate	Destroy tag identified by TID (The kill password must not be the default 00000000. If the kill password is set to default, it must be modified to a non-default password before destruction).		
parameter	name	type	Remark
	TID	String	Tag's TID, in hexadecimal string format.
	password	String	Kill Password, 2 words in length, hexadecimal string format.
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.6.11 writeTagEpcByTid [Write EPC bank by TID]

definition	int writeTagEpcByTid (String TID , String password , String writeData);		
illustrate	Modify the tag's EPC value via TID, rewriting the current EPC to the specified "writeData" EPC value, which will automatically update the PC value.		
parameter	name	type	Remark
	TID	String	Tag's TID, in hexadecimal string format.
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	writeData	String	Data to be written, in hexadecimal string format. The data length must be an integer multiple of 1 Word length (i.e., the hexadecimal string length must be a multiple of 4).
return (int)	Success: RfidErrorConstants.SUCCESS; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.6.12 maskReadTag [Read tag with mask]

definition	TagResult maskReadTag (int m emBank , int startAdd , int wordCnt , String password , int memMask , int startMask , int lenMask , String datasMask);
illustrate	Read tag content with mask

parameter	name	type	Remark
	memBank	int	Memory banks of the tag to be read: MemoryBank.RESERVED (RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	The starting position of the memory bank to be read/written, in word address . As unit.
	wordCnt	int	Word Count to Read (When reading large content, iterative reads are recommended with a maximum of 32 Words per operation).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	memMask	int	Filter bank: MemoryBank.RESERVED (RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startMask	int	Start Address Calculation: Note the parameter differs from other methods. Data indexing is based on character positions in the hexadecimal string: EPC Memory Bank Example EPC: E280FE9889895674 To mask "FE98" → Start Address = 12 (Reason: EPC includes CRC+PC prefix) TID/USER Memory Bank Example

			TID/USER: 00001234567898762345 To mask "56789876" → Start Address = 8 (No prefix, direct indexing from position 0)
	lenMask	int	Mask Length Parameter: Note this differs from other methods. The value specifies the character length of the target hexadecimal string to filter, and must be an even number: EPC Memory Bank Examples EPC: E280FE9889895674 To mask "FE98" → Length = 4 *(4 hex characters = 2 bytes)* EPC: 00001234567898762345 To mask "567898762345" → Length = 12 *(12 hex characters = 6 bytes)*
	datasMask	String	Filter data (even length)
return (String)	TagResult class field information: **code** : Status code: Success ; RfidErrorConstants.SUCCESS ; Failure: Other (For error code information, reference: <Appendix 1: Common Error Codes>) . **message** : Error message. **data** : If code == RfidErrorConstants.SUCCESS, this field returns the read tag value.		

Reference code	Tag content: Epc:123456789999 Tid:23456789000055557777 User:345678912 **Example 1:** **Operation Description:** Read EPC bank data (starting from Word Address 2, length: 3 Words), while applying a mask filter on the TID bank (starting from the 5th character (0-based index 4), matching 2 characters: "56"). **Parameter Breakdown:** RFIDSDKManager.getInstance().getRfidManager().maskReadTag(MemoryBank.EPC, // Target memory bank: EPC 2, // Start Word Address: 2 3, // Read length: 3 Words "00000000", // Access password (default) MemoryBank.TID, // Mask bank: TID 4, // Mask start position: 5th character (index 4, 0-based) 2, // Mask length: 2 characters "56" // Match value: "56") **Example 2:** **Operation Description:** Read EPC bank data (starting from Word Address 2, length: 3 Words), while applying a mask filter on the EPC bank itself

	(skip first 8 characters of CRC+PC, match "6789" starting from the 14th character (0-based index 13)). **Parameter Breakdown:** RFIDSDKManager.getInstance().getRfidManager().maskReadTag(MemoryBank.EPC, // Target memory bank: EPC 2, // Start Word Address: 2 3, // Read length: 3 Words "00000000", // Access password (default) MemoryBank.EPC, // Mask bank: EPC 13, // Mask start position: 14th character (index 13) 4, // Mask length: 4 characters "6789" // Match value: "6789")

2.6.13 maskWriteTag [Write tag with mask]

definition	int maskWriteTag (String writeData , int memBank , int startAdd , int wordCnt , String password , int memMask , int startMask , int lenMask , String datasMask);		
illustrate	Write the tag.		
parameter	name	type	Remark
	writeData	String	Data to be written, in hexadecimal string format. The data length must be an integer multiple of 1 Word

			length (i.e., the hexadecimal string length must be a multiple of 4).
	memBank	int	TAG storage bank : MemoryBank.RESERVED (RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	Starting position of the memory area to be read/written, in units of Word Address. Note that when writing to the EPC bank: 1>.If the EPC length needs to be changed, writing must start from the PC word (starting address 1). The calculated value of the PC word must be placed as the first word of the write data. (PC: Protocol Control) 2>.If the EPC length does not need to change after writing, writing may start from starting address 2 directly.
	wordCnt	int	Word Count to Read (When reading large content, iterative reads are

			recommended with a maximum of 32 Words per operation).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	memMask	int	Filter bank: MemoryBank.RESERVED (RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startMask	int	Start Address Calculation: Note the parameter differs from other methods. Data indexing is based on character positions in the hexadecimal string: EPC Memory Bank Example EPC: E280FE9889895674 To mask "FE98" → Start Address = 12 (Reason: EPC includes CRC+PC prefix) TID/USER Memory Bank Example TID/USER: 00001234567898762345 To mask "56789876" → Start Address = 8 (No prefix, direct indexing from position 0)
	lenMask	int	Mask Length Parameter: Note this differs from other methods. The

			value specifies the character length of the target hexadecimal string to filter, and must be an even number: EPC Memory Bank Examples EPC: E280FE9889895674 To mask "FE98" → Length = 4 *(4 hex characters = 2 bytes)* EPC: 00001234567898762345 To mask "567898762345" → Length = 12 *(12 hex characters = 6 bytes)*
	datasMask	String	Filter data (even length)
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	example: Tag Content: Epc:123456789999 Tid:23456789000055557777 User:34567891 **Example 1: Write EPC Bank + Filter TID Bank** **Operation Requirements:** Write data 999999999999 to EPC Bank (12 characters, requires 3-word length).		

Filter condition: Extract 2 characters from TID Bank starting at character 4 (index 3), matching value "56".

Parameter Conversion:

Filter start position: startMask=3 (index 3 corresponds to character 4).

Filter length: lenMask=2 (character count).

Code Implementation:

```
RFIDSDKManager.getInstance().getRfidManager().maskWriteTag(
    "999999999999",     // Data to write
    MemoryBank.EPC,     // Target memory bank (EPC)
    2,                    // Write start address (2-word offset = 8-character offset)
    3,                  // Write length (3 words = 12 characters)
    "00000000",         // Reserved parameter (fixed)
    MemoryBank.TID,     // Filter memory bank (TID)
    3,                          // Filter start index (character position)
    2,                  // Filter length (character count)
    "56"                // Filter match value
);
```

Example 2: Write USER Bank + Filter EPC Bank

Operation Requirements:

Write data 55555555 to USER Bank (8 characters, requires 2-word length).

Filter condition: Extract 2 characters from EPC Bank starting at character 14 (index 13), matching

	value "67". **Parameter Conversion:** Filter start position: startMask=13 (index 13 corresponds to character 14). Filter length: lenMask=2 (character count). **Code Implementation:** RFIDSDKManager.getInstance().getRfidManager().maskWriteTag("55555555", // Data to write MemoryBank.USER, // Target memory bank (USER) 0, // Write start address (0-word = USER bank start) 2, // Write length (2 words = 8 characters) "00000000", // Reserved parameter (fixed) MemoryBank.EPC, // Filter memory bank (EPC) 13, // Filter start index (character position) 2, // Filter length (character count) "67" // Filter match value);

2.6.14 readTagExt [Read large capacity tags]

definition	TagResult readTagExt (String EPC , int memBank , int startAdd , int wordCnt , String password);
illustrate	Read tag information.

parameter	name	type	Remark
	EPC	Stirng	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of the hexadecimal string must be a multiple of 4).
	memBank	int	TAG Memory Banks: MemoryBank.RESERVED (RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	Start Address of the target memory region for read/write operations, specified in Word Address units.
	wordCnt	int	Word Count to Read (When reading large content, iterative reads are recommended with a maximum of 32 Words per operation).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
return (String)	TagResult class field information: **code** : Status code: Success :RfidErrorConstants.SUCCESS ; Failure: Other (For error code information, reference: <Appendix 1: Common Error Codes>) . **message** : Error message. **data** : If code == RfidErrorConstants.SUCCESS, this field returns the read tag value.		
Reference	none		

code	

2.6.15 writeTagExt [Writing Large-Capacity Tags]

definition	int writeTagExt (String EPC , String password , int memBank , int startAdd , String writeData);		
illustrate	Writing Large-Capacity Tags.		
parameter	name	type	Remark
	EPC	String	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of the hexadecimal string must be a multiple of 4).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	memBank	int	TAG Memory Banks: MemoryBank.RESERVED (RESERVED) MemoryBank.EPC (EPC) MemoryBank.TID (TID) MemoryBank.USER (USER)
	startAdd	int	Starting position of the memory area to be read/written, in units of Word Address. Note that when writing to the EPC bank: 1>.If the EPC length needs to be

			changed, writing must start from the PC word (starting address 1). The calculated value of the PC word must be placed as the first word of the write data. (PC: Protocol Control) 2>.If the EPC length does not need to change after writing, writing may start from starting address 2 directly.
	writeData	String	Data to be written, in hexadecimal string format. The data length must be an integer multiple of 1 Word length (i.e., the hexadecimal string length must be a multiple of 4).
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.6.16 eraseTag [Erase Tag]

<table>
<tr><td>definition</td><td colspan="3">int eraseTag (String EPC , String password , int memBank , int startAdd , int wordCnt);</td></tr>
<tr><td>illustrate</td><td colspan="3">Erase tag information , the content of the erased data block will be set to 0 .</td></tr>
<tr><td rowspan="6">parameter</td><td>name</td><td>type</td><td>Remark</td></tr>
<tr><td>EPC</td><td>Stirng</td><td>Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of the hexadecimal string must be a multiple of 4).</td></tr>
<tr><td>password</td><td>String</td><td>Tag Access Password: 2 Words in length, hexadecimal string format.</td></tr>
<tr><td>memBank</td><td>int</td><td>TAG Memory Banks:
MemoryBank.RESERVED (RESERVED)
MemoryBank.EPC (EPC)
MemoryBank.TID (TID)
MemoryBank.USER (USER)</td></tr>
<tr><td>startAdd</td><td>int</td><td>The starting word address for erasure, in units of word address.</td></tr>
<tr><td>wordCnt</td><td>int</td><td>Erase word length.</td></tr>
<tr><td>return
(int)</td><td colspan="3">Success: RfidErrorConstants.SUCCESS ;
Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .</td></tr>
<tr><td>Reference code</td><td colspan="3">none</td></tr>
</table>

2.6.17 findEpc [Query Tag]

definition	Tag6C findEpc (String selectEpc);		
illustrate	Query Tag.		
parameter	name	type	Remark
	selectEpc	String	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of the hexadecimal string must be a multiple of 4).
return (int)	Detection is confirmed when Tag6C returns non-null. Its RSSI value provides signal strength metrics for proximity evaluation and spatial positioning analysis.		
Reference code	none		

2.6.18 lightUpLedTag [Light up the specified LED tag]

definition	int lightUpLedTag (String EPC , String password , int milliseconds);		
illustrate	Light up the specified LED tag		
parameter	name	type	Remark
	EPC	String	Tag EPC Data (excluding CRC and PC) must have a data length that is an integer multiple of 1 word (i.e. the length of

			the hexadecimal string must be a multiple of 4).
	password	String	Tag Access Password: 2 Words in length, hexadecimal string format.
	milliseconds	int	Parameter range: 100ms - 25500ms; must be passed in multiples of 100 ms. For example, to illuminate for the maximum 25500 ms, pass 25500.
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.6.19 startInventoryLed [Query ELD tag]

definition	int startInventoryLed (int manufacturers , List<String> epcs);		
illustrate	Illuminate the selected LED tags.		
parameter	name	type	Remark
	manufacturers	int	Manufacturer code(0 or 1) 0: Caravelle 1: Yilian

	epcs	List<String>	Set of EPC tags requiring illumination
return (int)	Success: RfidErrorConstants.SUCCESS; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.6.20 stopInventoryLed [Stop querying ELD tags]

definition	void stopInventoryLed ();		
illustrate	Deactivate LED illumination for tags.		
parameter	name	type	Remark
return (void)			
Reference code	none		

2. 7 Frequency band and other settings

2.7.1 setWorkRegion [Set regional frequency band]

definition	int setWorkRegion (CountryCode cntyCode);		
illustrate	Configure frequency bands according to regional settings. Note: If you want to set the region, Appendix 4: Regional Comparison Table You can refer to the API 2.7.3 setFrequencyRegion () Method to set.		
parameter	name	type	Remark
	cntyCode	CountryCode	International regional parameters (Appendix 4: Regional comparison table) For example: CountryCode.ALB CountryCode.BEL ...
return (int)	Success : RfidErrorConstants.SUCCESS ; Failure : Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int ret = RFIDSDKManager.getInstance().getRfidManager() .setWorkRegion(CountryCode.ALB);		

2.7.2 getWorkRegion [Get working frequency]

definition	CountryCode getWorkRegion ();
illustrate	Retrieve the regional frequency band configuration currently active on the device. A null return value indicates the region may not have been previously set via setWorkRegion(CountryCode cntyCode). You can use 2.7.4 getFrequencyRegion() method to get the current

	frequency band information.		
parameter	name	type	Remark
return (CountryCode)	Success : CountryCode is not null ; Failure : CountryCode is null ;		
Reference code			

2.7.3 setFrequencyRegion [Set frequency region]

definition	int setFrequencyRegion (FrequencyRegion region , int btStartIndex , int btEndIndex) ;		
illustrate	Set the frequency region (the frequencies supported by the reader by default).		
parameter	name	type	Remark

	region	FrequencyRegion	Frequency band type .
			FrequencyRegion.UKRAINE;//Ukraine band
			FrequencyRegion.CHINESE1;//Chinese band 1
			FrequencyRegion.EU;//EU band
			FrequencyRegion.CHINESE2;//Chinese band 2
			FrequencyRegion.US;//US band
			FrequencyRegion.KOREAN;//Korean band
			FrequencyRegion.PERU;//Peru band
			FrequencyRegion.US3;//US band3
			FrequencyRegion.Taiwan;//Taiwan band
			
			For details, reference: Appendix 2: Frequency Band Description

	btStartIndex	int	Starting frequency channel index of the spectrum.
	btEndIndex	int	Termination frequency channel index of the spectrum. Configures the operating range of the RF output spectrum.

			Rules: 1.Both start and end frequencies must reside within the regulatory-defined range. 2.The start frequency must be less than or equal to the end frequency. 3.If end frequency equals start frequency, single-frequency mode is enabled."**
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int ret = RFIDSDKManager.getInstance().getRfidManager().setFrequencyRegion (FrequencyRegion.US, FrequencyRegion.US.getMinChannelIndex(), FrequencyRegion.US.getMaxChannelIndex());		

2.7.4 getFrequencyRegion [Get frequency region]

definition	FrequencyRegion getFrequencyRegion ();		
illustrate	Get the frequency region.		
parameter	name	type	Remark
return (FrequencyRegion)	FrequencyRegion frequencyRegion = RFIDSDKManager.getInstance().getRfidManager().getFrequencyRegion (); 1. Get frequency band: `frequencyRegion.getbtRegion()`		

	2. Get start channel index: `frequencyRegion.getMinChannelIndex()` 3. Get end channel index: `frequencyRegion.getMaxChannelIndex()` 4. Calculate start frequency (MHz): float startFreq = frequencyRegion.getStartFreqMHz() + frequencyRegion.getMinChannelIndex() * frequencyRegion.getStepSizeMHz() 5. Calculate end frequency (MHz): float endFreq = frequencyRegion.getStartFreqMHz() + frequencyRegion.getMaxChannelIndex() * frequencyRegion.getStepSizeMHz() If frequencyRegion is null, it indicates a retrieval failure.
Reference code	

2.7.5 getFirmwareVersion [Get firmware version]

definition	String getFirmwareVersion ();		
illustrate	Get the firmware version of the RFID module .		
parameter	name	type	Remark
return (String)	Returns the RFID module firmware version information of String type		

Reference code	none

2.7.6 setTagFocus [Set TagFocus command]

definition	int setTagFocus (int enable);		
illustrate	This command is used to set the TagFocus tag and is limited to the Ex10 series.		
parameter	name	type	Remark
	enable	int	0-off; 1-on
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.7.7 getTagFocus [Read TagFocus command]

definition	int getTagFocus ();		
illustrate	This command is used to read the TagFocus tag and is limited to the Ex10 series.		
parameter	name	type	Remark
return (int)	Success: RfidErrorConstants.SUCCESS-off; 1- on ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.7.8 getModuleFirmware [Get module firmware information]

definition	String getModuleFirmware ();		
illustrate	Get module firmware information		
parameter	name	type	Remark
return (String)	Module Information		
Reference code	none		

2.7.9 GetReaderType [Get module type]

definition	int GetReaderType ();		
illustrate	Get module type		
parameter	name	type	Remark
return (int)	Success: greater than 0 Failure: -1.		
Reference code	none		

2.7.10 getReaderTemperature [Query internal temperature]

definition	String getReaderTemperature ();

illustrate	Check the internal temperature.		
parameter	name	type	Remark
return (String)	Returns the temperature value in string format (degrees Celsius)		
Reference code	none		

2.7.11 setCustomRegion [Set custom frequency band]

definition	int setCustomRegion (int flags , int FreSpace , int FreNum , int StartFre);		
illustrate	Set custom frequency bands ,		
parameter	name	type	Remark
	flags	int	Power-off configuration persistence flag 0: Retain settings 1: Discard settings
	int	FreSpace	Frequency interval (unit: kHz) , The parameter needs to be a multiple of 125。
	int	FreNum	Channel count
	int	StartFre	Starting frequency (unit:kHz)
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	RFIDSDKManager.getInstance().getRfidManager().setCustomRegion(0, 500,2,920000);		

2.7.12 getCustomRegion [Get custom frequency band]

definition	CustomRegionBean getCustomRegion ();		
illustrate	Get custom frequency band information; Note: After setting a custom frequency band, calling getFrequencyRegion() cannot get the relevant information of the standard frequency band.		
parameter	name	type	Remark

return (CustomRegionBean)	CustomRegionBean customRegion = rfidManager.getCustomRegion();
	if (customRegion != null) {
	// Extract parameters
	int band = customRegion.band[0]; // Frequency band region
	int stepKHz = customRegion.FreSpace[0]; // Frequency step (unit: kHz)
	int channelCount = customRegion.FreNum[0]; // Channel count
	int startFreqKHz = customRegion.StartFre[0]; // Starting frequency (unit: kHz)
	
	} else {
	System.err.println("Failed to retrieve custom frequency region configuration!");
	}		

Reference code	none		

2.7.13 getEx10Version [Obtain E-Series chip information]

definition	ExVersionInfo getEx10Version ();		
illustrate	retrieve the version and type of the employed RFID chip, exclusively excluding module version information.		
parameter	name	type	Remark

return (ExVersionInfo)	RFID chip information, including chip type and version number .
	
	ExVersionInfo Description :
	public class ExVersionInfo {
	public int code ; // Return value Success: RfidErrorConstants.SUCCESS Failure: Other
	public String versionInfo ; // Chip firmware version
	public int type ; //
	EXChipType.E310: E310;
	EXChipType.E510: E510;
	EXChipType.E710: E710;
	EXChipType.OTHER: Other
	}

Reference code	none

2.7.14 enableLog [enable or disable log]

definition	void enableLog (boolean enable);		
illustrate	Enable or disable logging.		
parameter	name	type	Remark
	enable	boolean	true: enable logging; false: disable logging
return (void)			
Referenc	none		

e code	

2.8 Buzzer setting

2.8.1 setBeepEnable [Set to enable the buzzer]

definition	int setBeepEnable (boolean enable);		
illustrate	Turn the buzzer on or off. It is off by default. If you want the buzzer to make a sound automatically after the RFID reads the tag, you need to turn it on.		
parameter	name	type	Remark
	enable	boolean	true - on , false - off
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2.8.2 setBeepVolume [Set the buzzer volume]

definition	int setBeepVolume (int volume);		
illustrate	Set the buzzer volume , volume defaults to 4, Note: The larger the value is, the higher the power consumption will be. Please set it according to the actual scenario or use the default value.		
parameter	name	type	Remark
	volume	int	Range 1-10 , default 4.

			1 is the smallest and 10 is the largest.
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	none		

2. 8 .3 startBeep [control the buzzer to sound once]

definition	void startBeep ();		
illustrate	Actively control the buzzer to make a sound .		
parameter	name	type	Remark
return (int)			
Reference code	none		

3 Firmware Upgrade

Note :

1>.RFID firmware upgrade: including RFID module upgrade and RFID chip upgrade;

2>. Firmware upgrade is an advanced operation, please operate with caution;

3>. Before updating, please confirm with our company the necessity of the upgrade and the latest firmware version to avoid upgrading to the wrong firmware.

Class used: FirmwareManager.getInstance();

3.1 All-in-one device upgrade (including RFG91 UART)

3.1.1 updateReaderFirmwareByFile [RFID module upgrade - by reading files]

definition	void updateReaderFirmwareByFile (Context context , FWUpdateCallback myCallback , String binName , String binPath);		
illustrate	To upgrade the module firmware, you need to have read and write permissions on the file. If the file is placed in the internal storage directory of the sdcard, the software needs to have management permissions for all files		
parameter	name	type	Remark
	context	Context	Context
	myCallback	FWUpdateCallback	Update process callback interface FWUpdateCallback Description:

			public abstract class FWUpdateCallback {
			/**
			* Upgrade result callback
			* @param res
			* 0: Upgrade successful
			* 11: Upgrade failed
			* 12: Upgrading
			* 13: Failed to enter upgrade mode
			* 14: Entering upgrade mode successfully
			* 20: Error reading file
			*/
			public abstract void updateResult(int res);
			
			/**
			* When upgrading, a callback will be made
			* @param progress The upgrade progress percentage returned during the upgrade process
			*/
			public abstract void updateProcess(int progress);
			}

	binName	String	The file name (without the suffix format) is used for comparison with the module model. When actually using it, you need to confirm the name with our company.
	binPath	String	The storage location of the firmware name. The application needs to have read and write

			permissions to this path. Path example: /storage/emulated/0/RRU7180M_V2.09.0.0.bin
return (int)			
Reference code	none		

3.1.2 updateReaderFirmwareByByte [RFID module upgrade--through byte stream]

definition	void updateReaderFirmwareByByte (Context context , FWUpdateCallback myCallback , String binName , byte[] fileBuff);		
illustrate	To upgrade the module firmware, you need the byte stream data corresponding to the firmware, which can be obtained from the local file		
parameter	name	type	Remark
	context	Context	Context
	myCallback	FWUpdate Callback	Update process callback interface FWUpdateCallback Description: public abstract class FWUpdateCallback { /** * Upgrade result callback * @param res * 0: Upgrade successful * 11: Upgrade failed

			* 12: Upgrading * 13: Failed to enter upgrade mode * 14: Entering upgrade mode successfully * 20: Error reading file */ public abstract void updateResult(int res); /** * When upgrading, a callback will be made * @param progress The upgrade progress percentage returned during the upgrade process */ public abstract void updateProcess(int progress); }
	binName	String	The file name (without the suffix format) is used for comparison with the module model. When actually using it, you need to confirm the name with our company.
	fileBuff	byte[]	Byte data stream read from the firmware file
return (int)			
Reference code	none		

3.1.3 updateEx10ChipFirmwareByFile [RFID chip upgrade - by reading files]

<table>
<tr><td>definition</td><td colspan="3">void updateEx10ChipFirmwareByFile (Context context , FWUpdateCallback myCallback , String binPath);</td></tr>
<tr><td>illustrate</td><td colspan="3">To upgrade the RFID chip firmware, you need to have read and write permissions on the file. If the file is placed in the internal storage directory of the sdcard, the software needs to have management permissions for all files</td></tr>
<tr><td rowspan="3">parameter</td><td>name</td><td>type</td><td>Remark</td></tr>
<tr><td>context</td><td>Context</td><td>Context</td></tr>
<tr><td>myCallback</td><td>FWUpdateCallback</td><td>Update process callback interface
FWUpdateCallback Description:
public abstract class FWUpdateCallback {
/**
* Upgrade result callback
* @param res
* 0: Upgrade successful
* 11: Upgrade failed
* 12: Upgrading
* 13: Failed to enter upgrade mode
* 14: Entering upgrade mode successfully
* 20: Error reading file
*/
public abstract void updateResult(int res);

/**
* When upgrading, a callback will be made
* @param progress The upgrade progress</td></tr>
</table>

			percentage returned during the upgrade process */ public abstract void updateProcess(int progress); }
	binPath	String	The storage location of the firmware name. The application needs to have read and write permissions to this path. Example path: /storage/emulated/0/ex10_appV2.01.02.bin
return (int)			
Reference code	none		

3.1.4 updateEx10ChipFirmwareByByte [RFID chip upgrade - through byte stream]

definition	void updateEx10ChipFirmwareByte (Context context , FWUpdateCallback myCallback , byte[] szbuff);		
illustrate	To upgrade the RFID chip firmware, you need the byte stream data corresponding to the firmware, which can be obtained from the local file		
parameter	name	type	Remark
	context	Context	Context

	myCallback	FWUpdate Callback	Update process callback interface
			FWUpdateCallback Description:
			public abstract class FWUpdateCallback {
			/**
			* Upgrade result callback
			* @param res
			* 0: Upgrade successful
			* 11: Upgrade failed
			* 12: Upgrading
			* 13: Failed to enter upgrade mode
			* 14: Entering upgrade mode successfully
			* 20: Error reading file
			*/
			public abstract void updateResult(int res);
			
			/**
			* When upgrading, a callback will be made
			* @param progress The upgrade progress percentage returned during the upgrade process
			*/
			public abstract void updateProcess(int progress);
			}

	szbuff	byte[]	Byte stream data read from the chip firmware file
return (int)			
Reference code	none		

3.2 Bluetooth device upgrade

3.2.1 updateBLEReaderFirmwareByFile [RFID module upgrade - by reading files]

definition	void updateBLEReaderFirmwareByFile (Context context , String address , FWUpdateCallback myCallback , String binName , String binPath);		
illustrate	To upgrade the module firmware, you need to have read and write permissions on the file. If the file is placed in the internal storage directory of the sdcard, the software needs to have management permissions for all files		
parameter	name	type	Remark
	context	Context	Context
	address	String	MAC address of the Bluetooth device
	myCallback	FWUpdate Callback	Update process callback interface FWUpdateCallback Description: public abstract class FWUpdateCallback { /** * Upgrade result callback * @param res * 0: Upgrade successful * 11: Upgrade failed * 12: Upgrading

			* 13: Failed to enter upgrade mode * 14: Entering upgrade mode successfully * 20: Error reading file */ public abstract void updateResult(int res); /** * When upgrading, a callback will be made * @param progress The upgrade progress percentage returned during the upgrade process */ public abstract void updateProcess(int progress); }
	binName	String	The file name (without the suffix format) is used for comparison with the module model. When actually using it, you need to confirm the name with our company.
	binPath	String	The storage location of the firmware name. The application needs to have read and write permissions to this path. Path example: /storage/emulated/0/RRU7180M_V2.09.0.0.bin
return (int)			
Reference	none		

code	

3.2.2 updateBLEReaderFirmwareByByte [RFID module upgrade--by reading files]

definition	void updateBLEReaderFirmwareByte (Context context , String address , FWUpdateCallback myCallback , String binName , byte[] fileBuff);		
illustrate	To upgrade the module firmware, you need the byte stream data corresponding to the firmware, which can be obtained from the local file		
parameter	name	type	Remark
	context	Context	Context
	address	String	MAC address of the Bluetooth device
	myCallback	FWUpdateCallback	Update process callback interface FWUpdateCallback Description: public abstract class FWUpdateCallback { /** * Upgrade result callback * @param res * 0: Upgrade successful * 11: Upgrade failed * 12: Upgrading * 13: Failed to enter upgrade mode * 14: Entering upgrade mode successfully * 20: Error reading file */ public abstract void updateResult(int res);

			/** * When upgrading, a callback will be made * @param progress The upgrade progress percentage returned during the upgrade process */ public abstract void updateProcess(int progress); }
	binName	String	The file name (without the suffix format) is used for comparison with the module model. When actually using it, you need to confirm the name with our company.
	fileBuff	byte[]	Byte stream data read from the firmware file
return (int)			
Reference code	none		

3.2.3 updateBLEEx10ChipFirmwareByFile [RFID module upgrade - through byte stream]

definition	void updateBLEEx10ChipFirmwareByFile (Context context , String address , FWUpdateCallback myCallback , String binName , String binPath);
illustrat	To upgrade the RFID chip firmware, you need to have read and

<table>
<tr><td>e</td><td colspan="3">write permissions on the file. If the file is placed in the internal storage directory of the sdcard, the software needs to have management permissions for all files</td></tr>
<tr><td rowspan="4">parameter</td><td>name</td><td>type</td><td>Remark</td></tr>
<tr><td>context</td><td>Context</td><td>Context</td></tr>
<tr><td>address</td><td>String</td><td>MAC address of the Bluetooth device</td></tr>
<tr><td>myCallback</td><td>FWUpdate Callback</td><td>Update process callback interface
FWUpdateCallback Description:
public abstract class FWUpdateCallback {
/**
* Upgrade result callback
* @param res
* 0: Upgrade successful
* 11: Upgrade failed
* 12: Upgrading
* 13: Failed to enter upgrade mode
* 14: Entering upgrade mode successfully
* 20: Error reading file
*/
public abstract void updateResult(int res);

/**
* When upgrading, a callback will be made
* @param progress The upgrade progress percentage returned during the upgrade process
*/
public abstract void updateProcess(int progress);
}</td></tr>
</table>

	binName	String	The file name (without the suffix format) is used for comparison with the module model. When actually using it, you need to confirm the name with our company.
	binPath	String	The storage location of the chip firmware name, you need to pass the absolute path, for example: /storage/emulated/0/ex10_appV2.01.02.bin
return (int)			
Reference code	none		

3.2.4 updateBLEEx10ChipFirmwareByByte [RFID module upgrade--through byte stream]

definition	void updateBLEEx10ChipFirmwareByte (Context context , String address , FWUpdateCallback myCallback , String binName , byte[] fileBuff);		
illustrate	To upgrade the RFID chip firmware, you need the byte stream data corresponding to the firmware, which can be obtained from the local file		
parameter	name	type	Remark
	context	Context	Context
	address	String	MAC address of the Bluetooth device
	myCallback	FWUpdate	Update process callback interface

		Callback	FWUpdateCallback Description:
			public abstract class FWUpdateCallback {
			/**
			* Upgrade result callback
			* @param res
			* 0: Upgrade successful
			* 11: Upgrade failed
			* 12: Upgrading
			* 13: Failed to enter upgrade mode
			* 14: Entering upgrade mode successfully
			* 20: Error reading file
			*/
			public abstract void updateResult(int res);
			
			/**
			* When upgrading, a callback will be made
			* @param progress The upgrade progress percentage returned during the upgrade process
			*/
			public abstract void updateProcess(int progress);
			}

	binName	String	The file name (without the suffix format) is used for comparison with the module model. When actually using it, you need to confirm the name with our company.
	fileBuff	byte[]	Byte data stream read from the firmware file

return (int)	
Reference code	none

4. Bluetooth and UART modular device interface description (non-integrated device)

Applicable devices: This section applies exclusively to RFG91 UART modular devices and Bluetooth devices;

Functionality: Includes barcode scan result retrieval, device status callback monitoring, device information acquisition, parameter configuration, etc.

Conditions of use: Normal operation of these interfaces depends on successful initialization as described in **[Section 2.1 Initialization and Connection].**

Class used: GripDeviceManager.getInstance();

4.1 Obtaining the results of scanning the barcode

Receive the scanned barcode result.

registerBarcodeCallback [Register the callback for scanning barcodes with the scanner head]

definition	void registerBarcodeCallback (BarcodeCallback cb);		
illustrate	Register the barcode scan callback.		
parameter	name	type	Remark
	cb	BarcodeCallback	Callbacks
return (void)	none		
Reference code	GripDeviceManager.getInstance().registerBarcodeCallback(new BarcodeCallback() {		

	@Override
	public void onResult(byte[] barcode) {
	// scanned barcode result
	}
	});

4.2 Device status callback monitoring

4.2.1 setKeyEventListener [Register key event listener]

definition	void setKeyEventListener (KeyEventListener cb);		
illustrate	Register callback (this method monitors corresponding events when the scan button is pressed or released). By default, the RFG91 button does not trigger RFID operations. To activate RFID functions via the RFG91 button, you must listen to this callback event and execute appropriate interface calls to trigger RFID actions.		
parameter	name	type	Remark

	cb	KeyEventListener	Button status callback:
			public void event(int keyCode, boolean isDown)
			{
			//keyCode== BTKeyEvent.BT_SCAN The handle button was pressed
			//isDown true: down false: up

			}
return (void)	none		

Reference code	GripDeviceManager.getInstance().setKeyEventListener(new KeyEventListener()
	{
	@Override
	public void event(int keyCode, boolean isDown) {
	//keyCode== BTKeyEvent.BT_SCAN The handle button was pressed
	//isDown true: down false: up
	if (keyCode == BTKeyEvent.BT_SCAN) {
	if (isDown) {
	//start Inventory
	//RFIDSDKManager.getInstance().getRfidManager().startInventory(1);
	} else {
	//stop Inventory
	//RFIDSDKManager.getInstance().getRfidManager().stopInventory();
	}
	}
	}
	});		

4.2.2 setBatteryGripListener [register device charging status, battery charge change monitoring]

definition	void setBatteryGripListener (BatteryGripListener cb);
illustrate	Register a callback for device battery status. This method monitors changes in charging state and battery level, triggering

<table>
<tr><td></td><td colspan="3">callbacks when values change. Since callbacks only occur during state transitions, to immediately obtain the current charging status and battery level, use the 4.5 getIsCharging() and 4.4 getElectricQuantity() methods.</td></tr>
<tr><td>parameter</td><td>name</td><td>type</td><td>Remark</td></tr>
<tr><td></td><td>cb</td><td>BatteryGripListener</td><td>Device charging status and battery level callback::
new BatteryGripListener() {
@Override
public void isChange(int change) {
//change 0: not charged 1: charged
}
@Override
public void level(int percent) {
//percent Power percentage 0-100%
}
};</td></tr>
<tr><td>return
(void)</td><td colspan="3">none</td></tr>
<tr><td>Reference code</td><td colspan="3">GripDeviceManager.getInstance().setBatteryGripListener(new
BatteryGripListener() {
@Override
public void isChange(int change) {
//change 0: not charged 1: charged
}
@Override
public void level(int percent) {
//percent Power percentage 0-100%
}</td></tr>
</table>

	});

4.3 setSwitchCallback [Monitor the switching between RFID modules]

definition	void setSwitchCallback (ModelInfoCallback cb);		
illustrate	Use class: RFIDSDKManager.getInstance() ; Callback for monitoring RFID module status changes when switching between DT610's built-in RFID and RFG91 UART devices. Note: Only effective when using DT610 with RFG91 UART devices. This method has no effect on other device combinations.		
parameter	name	type	Remark

	cb	ModelInfoCallback	Module information callback after switching:
			interface ModelInfoCallback {
			/**
			* @param type 0: built-in module (DT610 built-in module) 1: UART module (RFG91 UART device)
			* @param firmware: module version
			* @param power: power value used
			*/
			void onInfo(int type, String firmware, int power);
			
			/**

			* @param status
			* ModelStatus.Connected = 0; //Closed
			* ModelStatus.Separate = 1; //Separate
			* ModelStatus.StartingUp = 2; //Standby
			* ModelStatus.Shutdown = 3; //Shutdown
			*/
			void onModuleStatus(int status);
			
			}

return (void)	none		

Reference code	RFIDSDKManager.getInstance().setSwitchCallback(new ModelInfoCallback() {
	@Override
	public void onModuleStatus(int status) {
	}
	@Override
	public void onInfo(int type, String firmware, int power) {
	}
	});		

4.3 startScanBarcode [Start scanning barcode]

definition	int startScanBarcode (boolean enable);		
illustrate	Start scanning the Tag.		
parameter	name	type	Remark
	enable	boolean	true: trigger to start scanning false: Trigger to stop scanning

return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .
Reference code	none

4.4 getElectricQuantity [Get battery power]

definition	int getElectricQuantity();		
illustrate	Get battery level		
parameter	name	type	Remark
return (int)	Success: greater than or equal to 0; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int value = GripDeviceManager.getInstance().getElectricQuantity();		

4.5 getIsChange [Retrieve device charging status.]

definition	int getIsChange();		
illustrate	Retrieve device charging status.		
parameter	name	type	Remark
return (String)	Success: 0: Not charging 1: Charging; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		

Reference code	int value = GripDeviceManager.getInstance().getIsChange();

4.6 getVersionSystem [Get device version]

definition	String getVersionSystem();		
illustrate	Get the device version		
parameter	name	type	Remark
return String	Returns the read content if successful, otherwise returns null.		
Reference code	String sysVer = GripDeviceManager.getInstance().getVersionSystem();		

4.7 getVersionBLE [Get Bluetooth version]

definition	String getVersionBLE();		
illustrate	Get the device's Bluetooth firmware version		
parameter	name	type	Remark
return String	Returns the read content if successful, otherwise returns null.		
Reference	String bleVer =		

code	GripDeviceManager .getInstance().getVersionBLE();

4.8 getDeviceSN [Get device SN]

definition	String getDeviceSN();		
illustrate	Get device SN		
parameter	name	type	Remark
return String	Returns the read content if successful, otherwise returns null.		
Reference code	String sn = GripDeviceManager .getInstance().getDeviceSN();		

4.9 setBeepRange [Set buzzer volume]

definition	int setBeepRange(int beep);		
illustrate	Set the buzzer volume		
parameter	name	type	Remark
	beep	int	Range: 0-10 If it is 0, turn off the buzzer.
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		

Reference code	int ret = GripDeviceManager .getInstance().setBeepRange(10);

4.10 getBeepRange [Get the buzzer volume]

definition	int getBeepRange();		
illustrate	Get the buzzer volume.		
parameter	name	type	Remark
return (int)	Returns the volume of the buzzer (if it is less than 0, the acquisition fails)		
Reference code	int beepValue = GripDeviceManager .getInstance().getBeepRange();		

4.11 setSleepTime [Set sleep time]

definition	int setSleepTime(int time);		
illustrate	Set the sleep time, unit: seconds		
parameter	name	type	Remark
	time	int	Range: 1-3600
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int ret = GripDeviceManager .getInstance().setSleepTime(6 0);		

4.12 getSleepTime [Get sleep time]

definition	int getSleepTime();		
illustrate	Get the sleep time of the current device.		
parameter	name	type	Remark
return (int)	Returns the sleep time (failed to get if less than 0)		
Reference code	int sleep Value = GripDeviceManager .getInstance().getSleepTime();		

4. 1 3 setPowerOffTime [Set the timeout shutdown time]

definition	int setPowerOffTime(int offTime);		
illustrate	Set the time for timeout shutdown. If no interface call is made to the device or the physical button on the device is pressed, the device will shut down after the corresponding timeout. Unit: seconds		
parameter	name	type	Remark
	beep	int	Range: 1-10
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int ret = GripDeviceManager .getInstance().setBeepRange(10);		

4.14 getPowerOffTime [Get buzzer volume]

definition	int getPowerOffTime();		
illustrate	Get the shutdown timeout period.		
parameter	name	type	Remark
return (int)	Returns the shutdown timeout (failed to get if less than 0)		
Reference code	int OFF Value = GripDeviceManager .getInstance().getPowerOffTime();		

4.15 getBLEMac [Get the MAC address of the Bluetooth device]

definition	String getBLEMac();		
illustrate	Get the MAC address of the Bluetooth device, only for Bluetooth devices.		
parameter	name	type	Remark
return String	Success : Returns the MAC address, for example: 41:BB:00:F8:DB:51 Failure: Returns empty .		
Reference code	String mac = GripDeviceManager .getInstance().getBLEMac();		

4.16 setOfflineModeOpen [Set offline mode]

definition	int setOfflineModeOpen(int mode);		
illustrate	Set offline mode.		
parameter	name	type	Remark
	mode	int	Range: 0-1 0: Disable offline mode 1: Enable offline mode
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int ret = GripDeviceManager .getInstance().setOfflineModeOpen(1);		

4.17 getOfflineModeOpen [Get the offline mode open status]

definition	int getOfflineModeOpen();		
illustrate	Get offline mode status.		
parameter	name	type	Remark
return (int)	Return to offline mode state 0: Offline mode is off 1: Offline mode is on		
Reference code	int value = GripDeviceManager .getInstance().getOfflineModeOpen();		

4.18 setOfflineTransferClearData [Set whether to automatically clear data after offline mode transfer]

definition	int setOfflineTransferClearData(int mode);		
illustrate	Set whether to automatically clear the cache in the device after data transfer in offline mode.		
parameter	name	type	Remark
	mode	int	Range: 0-1 0: Do not clear automatically 1: Automatically clear
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int ret = GripDeviceManager .getInstance().setOfflineTransferClearData(1);		

4.19 getOfflineTransferClearData [Get the status of automatic data clearing in offline mode]

definition	int getOfflineTransferClearData();		
illustrate	The mode status of whether to automatically clear the data after obtaining offline data transmission.		
parameter	name	type	Remark

return (int)	Returns the status of data clearing in offline mode 0: Do not automatically clear data after offline data transmission 1: Automatically clear data after offline data transmission
Reference code	int value = GripDeviceManager .getInstance().getOfflineTransferClearData();

4 . 2 0 setOfflineTransferDelay [Set the transmission time interval between data in offline mode]

definition	int setOfflineTransferDelay(int delay);		
illustrate	Set the interval time for transmitting each piece of data in offline mode.		
parameter	name	type	Remark
	delay	int	Range: 0-10000ms
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int ret = GripDeviceManager .getInstance().setOfflineTransferDelay(0);		

4.2 1 getOfflineTransferClearData [Get the status of automatic data clearing in offline mode]

definition	int getOfflineTransferClearData();
illustrat	The mode status of whether to automatically clear the data after

e	obtaining offline data transmission.		
parameter	name	type	Remark
return (int)	If the result is greater than or equal to 0, it is successful, in milliseconds If it is less than 0, the acquisition fails.		
Reference code	int value = GripDeviceManager .getInstance().getOfflineTransferDelay();		

4.22 getOfflineQueryNum [Get the number of offline data]

definition	int[] getOfflineQueryNum();		
illustrate	Get the number of RFID tags and barcodes read offline.		
parameter	name	type	Remark
return (int[])	Return collection: int[] numArr If the length of numArr is greater than 0, it means the acquisition is successful. numArr[0]: Number of cached RFIDs numArr[1]: Number of cached scanned barcodes If numArr is equal to null or length is equal to 0, the acquisition fails.		
Reference code	int [] numArr = GripDeviceManager .getInstance().getOfflineQueryNum();		

4.23 getOfflineQueryMem [Get the memory percentage occupied by offline data]

definition	double [] getOfflineQueryMem();		
illustrate	Get the storage space ratio occupied by the scanned data in offline state.		
parameter	name	type	Remark

return (int)	Return collection: double[] memArr If the length of memArr is greater than 0, it means the acquisition is successful. memArr[0]: The proportion of cached RFID memArr[1]: The percentage of cached scanned barcodes If memArr is equal to null or length is equal to 0, the acquisition fails.
Reference code	double[] memArr = GripDeviceManager .getInstance().getOfflineQueryMem();

4.24 offlineManaulClearData [Actively clear data saved in offline mode]

definition	int offlineManaulClearData();		
illustrate	Automatically clear data saved in offline mode.		
parameter	name	type	Remark
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int value = GripDeviceManager .getInstance().offlineManaulClearData();		

4.25 offlineStartTransferRFID [Actively clear RFID data saved in offline mode]

definition	int offlineStartTransferRFID();
illustrate	Start transferring data saved in offline data. The data is uniformly called back through the RFID's IRfidCallback callback function.

parameter	name	type	Remark
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int ret = GripDeviceManager .getInstance().offlineStartTransferRFID();		

4.26 offlineStartTransferScan [Actively clear RFID data saved in offline mode]

definition	int offlineStartTransferScan();		
illustrate	Start transferring data saved in offline data. The data is called back through the BarcodeCallback callback function.		
parameter	name	type	Remark
return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .		
Reference code	int ret = GripDeviceManager .getInstance().offlineStartTransferScan();		

4.27 modeResetFactory [Device firmware restores factory settings]

definition	int modeResetFactory();		
illustrate	Reset the device's firmware settings (not Bluetooth firmware and RFID firmware) .		
parameter	name	type	Remark

return (int)	Success: RfidErrorConstants.SUCCESS ; Failure: Others (For error code information, reference: <Appendix 1: Common Error Codes>) .
Reference code	int ret = GripDeviceManager .getInstance().modeResetFactory();

Appendix 1: Common error codes

Error Code (RfidErrorConstants)	describe
SUCCESS	Operation successful
TAG_NOT_FOUND	No RFID tag detected in field
INSUFFICIENT_POWER	Tag power insufficient for memory write operation
MEMORY_ERROR	Memory out-of-bounds or unsupported PC value
MEMORY_LOCKED	Memory bank permanently locked, write prohibited
ACCESS_PWD_ERROR	Access password incorrect
KILL_PWD_ERROR	Kill password incorrect
INVALID_KILL_PWD	Kill password cannot be all zeros
UNSUPPORTED_CMD	Tag does not support this command
INVALID_ACCESS_PWD	Access password required for this command
READ_PROTECTED	Tag already read-protected
NOT_READ_PROTECTED	Tag not read-protected, unlock not required
GENERIC_ERROR	Non-specific tag error (no detailed code)
WRITE_FAILED	Write failed due to locked memory blocks
LOCK_FAILED	Memory lock operation failed
ALREADY_LOCKED	Memory already permanently locked
PARAM_SAVE_FAILED	Parameter save failed (volatile until reader power-off)
ADJUST_FAILED	Parameter adjustment failed
ANTENNA_ERROR	Antenna detection failure
CMD_EXEC_FAILED	Command execution error
COMMUNICATION_FAILURE	Tag detected but communication unstable
NO_TAG	No operable RFID tag in field
TAG_ERROR	Tag returned error code
INVALID_CMD_LENGTH	Invalid command length
ILLEGAL_CMD	Illegal command for current tag state
PARAM_ERROR	Invalid command parameters
INVALID_INVENTORY_MODE	Inventory mode not in custom configuration

UNSUPPORTED_FEATURE	Firmware does not support this feature
COMMUNICATION_ERROR	RF communication error
SDK_UNINITIALIZED	SDK not initialized
DEVICE_DISCONNECTED	RFID reader not connected (external/BT device)
SEND_CMD_FAILED	Command transmission failed (external/BT device)

The error code can also be used to get error information in the following ways:

```
    /**
* Get the error code corresponding to the error information
* @param errorCode error code
* @return errorMessage returns specific error message
*/
String errorMessage = RfidErrorConstants.getDescription(int errorCode);
```

Appendix 2: Frequency band description

European standard frequency band (865~868MHz)

Frequency region	Frequency band name	Number of frequency points	Channel spacing kHz	Frequency formula MHz	Range (MHZ)
FrequencyRegion.UKRAINE	Ukraine	7	100	868.0 + N*0.1(MHz) where N∈[0,6]	868.0-868.6
FrequencyRegion.CHINESE1	China Zone 1	20	250	840.125 + N * 0.25 (MHz) where N∈[0,19]	840.125-844.875
FrequencyRegion.EU	European Union	4	600	865.7 + N * 0.6(MHz) where N∈[0,3]	865.7-867.5

US standard frequency band (902~928MHz)

Frequency region	Frequency band name	Number of frequency points	Channel spacing kHz	Frequency formula MHz	Range (MHZ)
FrequencyRegion.CHINESE2	China Zone 2	20	250	920.125 + N * 0.25 (MHz) where N∈[0,19]	920.125-924.875
FrequencyRegion. US	United States	50	500	902.75 + N * 0.5 (MHz) where N∈[0,49]	902.75-927.25
FrequencyRegion	South	6	6	917. 3 + N *	917.3-920.3

.KOREAN	Korea		00	0. 6 (MHz) where N∈[0, 5]	
FrequencyRegion .PERU	Peru	12	900	916.2 + N*0.9(MHz) where N∈[0, 11]	916.2-926.1
FrequencyRegion . US3	USA Extended	53	500	902 + N * 0.5 (MHz) where N ∈[0,52]	902-928
FrequencyRegion .Taiwan	Taiwan, China	12	500	922.25 + N * 0.5 (MHz) where N∈ [0,11]	922.25-927.75

Examples of setting these region specific bands:
For example, European Union standard FrequencyRegion.EU : (from top to bottom, corresponding to parameters 0-3)
865.7 0
866.3 1
866.9 2
867.5 3
If you need to set the European Union standard 866.3-866.9
mRfidManager.setFrequencyRegion(FrequencyRegion.EU , 1 , 2);
If you need to set the European Union standard 865.7-867.5
mRfidManager.setFrequencyRegion(FrequencyRegion.EU , 0 , 3);

Appendix 3: RSSI parameter comparison table

RSSI Parameters	Signal Strength	RSSI Parameters	Signal Strength
110(0x6E)	-19dBm	70(0x46)	-60dBm
109(0x6D)	-20dBm	69(0x45)	-61dBm
108(0x6C)	-21dBm	68(0x44)	-62dBm
107(0x6B)	-22dBm	67(0x43)	-63dBm
106(0x6A)	-23dBm	66(0x42)	-64dBm
105(0x69)	-24dBm	65(0x41)	-65dBm
104(0x68)	-25dBm	64(0x40)	-66dBm
103(0x67)	-26dBm	63(0x3F)	-67dBm
102(0x66)	-27dBm	62(0x3E)	-68dBm
101(0x65)	-28dBm	61(0x3D)	-69dBm
100(0x64)	-29dBm	60(0x3C)	-70dBm
99(0x63)	-30dBm	59(0x3B)	-71dBm
98(0x62)	-31dBm	58(0x3A)	-72dBm
97(0x61)	-32dBm	57(0x39)	-73dBm
96(0x60)	-33dBm	56(0x38)	-74dBm
95(0x5F)	-34dBm	55(0x37)	-75dBm
94(0x5E)	-35dBm	54(0x36)	-76dBm
93(0x5D)	-36dBm	53(0x35)	-77dBm
92(0x5C)	-37dBm	52(0x34)	-78dBm
91(0x5B)	-38dBm	51(0x33)	-79dBm
90(0x5A)	-39dBm	50(0x32)	-80dBm
89(0x59)	-41dBm	49(0x31)	-81dBm
88(0x58)	-42dBm	48(0x30)	-82dBm
87(0x57)	-43dBm	47(0x2F)	-83dBm
86(0x56)	-44dBm	46(0x2E)	-84dBm
85(0x55)	-45dBm	45(0x2D)	-85dBm
84(0x54)	-46dBm	44(0x2C)	-86dBm
83(0x53)	-47dBm	43(0x2B)	-87dBm
82(0x52)	-48dBm	42(0x2A)	-88dBm
81(0x51)	-49dBm	41(0x29)	-89dBm
80(0x50)	-50dBm	40(0x28)	-90dBm
79(0x4F)	-51dBm	39(0x27)	-91dBm

78(0x4E)	-52dBm	38(0x26)	-92dBm
77(0x4D)	-53dBm	37(0x25)	-93dBm
76(0x4C)	-54dBm	36(0x24)	-94dBm
75(0x4B)	-55dBm	35(0x23)	-95dBm
74(0x4A)	-56dBm	34(0x22)	-96dBm
73(0x49)	-57dBm	33(0x21)	-97dBm
72(0x48)	-58dBm	32(0x20)	-98dBm
71(0x47)	-59dBm	31(0x1F)	-99dBm

illustrate:

It can be converted to a dBm value using a utility class :

```
int dBm = RssiToDBmUtils.dBmConver(int RSSI);
```

Alternatively, configure the default output format using method 2.5.15 setRssiInDbm(boolean isDbm).

Appendix 4: Regional Comparison Table

parameter(CountryCode)	bank
ALB	Albania
ARG	Argentina
ARM	Armenia
AUT	Austria
AZE	Azerbaijan
BLR	Belarus
BEL	Belgium
BIH	Bosnia and Herzegovina
BRA	Brazil
BUL	Bulgaria
CAN	Canada
CHI	Chile
CHN	China
COL	Colombia
CRC	Costa Rica
CRO	Croatia
CYP	Cyprus
CZ	Czech Republic
DEN	Denmark
DOM	Dominican Republic
EST	Estonia
FIN	Finland
FRA	France
GER	Germany
GRE	Greece
HUN	Hungary
ISL	Iceland
IRI	Iran, Islamic Rep.
ITA	Italy
KOR	Korea, Rep.
LIE	Liechtenstein
LTU	Lithuania
LUX	Luxembourg
MKD	Macedonia, FYR
MLT	Malta
MEX	Mexico
MDA	Moldova

NED	Netherlands
NZD	New Zealand
NGR	Nigeria
NOR	Norway
PAN	Panama
PER	Peru
POL	Poland
POR	Portugal
ROM	Romania
RUS	Russian Federation
KSA	Saudi Arabia
SVK	Slovak Republic
SLO	Slovenia
RSA	South Africa
ESP	Spain
SWE	Sweden
SUI	Switzerland
HKG	Hong Kong
THA	Thailand
TUN	Tunisia
TUR	Turkey
UAE	United Arab Emirates
GBR	United Kingdom
USA	United States
URU	Uruguay
VEN	Venezuela, RB
UKR	Ukraine
MAS	Malaysia
SIN	Singapore
AUS	Australia
IND	India
VIE	Vietnam
ISR	Israel
INA	Indonesia
JPN	Japan
PHI	Philippines

Appendix 5: APP packaging obfuscation rules

```
#RFID-SDK obfuscation rules
-keep class com.rfid.**{*;}
-keep class com.rfiddevice.**{*;}
-keep class com.android.hw.**{*;}
-keep class com.ubx.**{*;}
-keep class com.iflytek.**{*;}
-keep class com.lib.**{*;}
-keep class android.device.**{*;}
-keep class android.content.**{*;}
-keep class android.os.**{*;}
```

Remark

Additional API Notes

1. Long-distance (with handle) device controls the scanning head to emit light (for non-Bluetooth devices)

RFIDSDKManager.getInstance().enableScanHead(boolean isOpen);

Note: This method is for long-distance RFID devices with handles. It controls th e buttons on the handles to trigger the code scanning at the terminal.

parameter	type	illustrate
isOpen	boolean	true, the controller button triggers the code scan; false, the controller button does not trigger the code scan

2. When using NFC to connect to a Bluetooth device, the information read is used to resolve the MAC address

String mac = NFCUtils.getBleMac(NdefMessage ndefMessage);

NdefMessage object obtained by NFC . The method returns the MAC address of th e Bluetooth device. After obtaining the address, you can pass it to the mac param eter in the 2.1.2.1 initBTToMac (Context context, String mac, InitListener listener); method.

parameter	type	illustrate
ndefMessage	NdefMessage	NdefMessage object obtained by NFC

suggestion

- The init() method and the release() method appear in pairs. When exiting the application, call release() to disconnect the RFID connection .

- Perform troubleshooting according to the error code comparison table .

Contact & Support

For the latest updates, sample code, and technical support, please contact our support team.

End of document .